

A COMPLETE, PLAIN-ENGLISH GUIDE

The Options Market-Making & Multi-Model Delta-Hedging Simulator

Every formula, every workflow, and every dashboard tab explained from first principles — no finance or programming background assumed.

Black-Scholes · Heston · Merton

Delta & Delta-Vega Hedging

CVaR & Extreme Value Theory

Bootstrap Statistics

American Options (Trees & PDEs)

Monte Carlo & Variance Reduction

16-Tab Interactive Dashboard

Companion document to `README.md` and `report/report.md`. This guide assumes you are learning options, hedging, and quantitative risk management for the first time, and walks through the reasoning behind every piece of code in `src/` and every tab of `dashboard.py` step by step.

Table of Contents

Part 1 — How to Read This Guide

Part 2 — Foundations — Options, Hedging, and Why Any of This Matters

- What is an option, really?
- What does a market maker do?
- What does it mean to "hedge"?
- Volatility, drift, and randomness
- Why "jumps" are the whole point of this project

Part 3 — The Three Pricing Models

- Black-Scholes-Merton: the baseline model
- The Greeks: Delta, Gamma, Vega, Theta, Rho
- Heston: when volatility itself is random
- The COS method: making Heston fast
- Merton: when prices can jump

Part 4 — Calibration — Teaching the Models About the Real World

Part 5 — The Hedging Engine — Simulating a Market Maker's Life

- The replication argument and the cash account
- Fixed-frequency rebalancing
- Band ("no-transaction region") hedging
- Delta-Vega hedging with a second option

Part 6 — Measuring Risk — VaR and CVaR From Scratch

Part 7 — How Sure Are We? Bootstrap Statistics and Hypothesis Testing

Part 8 — Extreme Value Theory — Estimating Risks You Haven't Seen Yet

Part 9 — American Options — Binomial Trees and the Crank-Nicolson PDE

Part 10 — Monte Carlo Simulation and Variance Reduction

Part 11 — P&L Attribution, Stress Testing, and Model Validation

Part 12 — The Experiment Pipeline — A 14-Step Walkthrough

Part 13 — Inside the Dashboard — All 16 Tabs Explained

Part 14 — Headline Findings, Glossary, and References

PART 1

How to Read This Guide

A short note on who this is for, and the visual language used throughout — read this first, it will save you time later.

This document explains, from the ground up, a project that answers one question: *when a market maker sells an option and hedges it, how much extra risk are they secretly carrying if the model they use to hedge is too simple for how the real market actually behaves?* That single question turns out to require a genuinely large toolbox — option pricing theory, statistics, numerical methods, and software engineering — and this guide walks through every piece of that toolbox as if you are encountering it for the very first time.

You do not need to know what an option is, what "volatility" means in finance, or how to read a formula with Greek letters in it. Every technical term is defined the first time it is used, every formula is broken down symbol by symbol, and most ideas are illustrated with a small worked example using real numbers before we generalize. If you already know some of this material, feel free to skip ahead using the Table of Contents — every internal link is clickable.

The four boxes you will see throughout this guide

PLAIN ENGLISH

This box translates whatever we just introduced into an everyday analogy, with no jargon and no formulas. If you only have time to read one box per topic, read this one.

FORMULA

This box shows the actual mathematical definition, with every single symbol explained immediately underneath it in a small table. Nothing is left as "obvious" — if a symbol appears, it is defined.

WORKED EXAMPLE

This box plugs in real, concrete numbers (often the actual numbers this project produced for SPY or Bitcoin) so you can see the formula actually being used, not just stated abstractly.

WHERE THIS LIVES IN THE PROJECT

This box tells you the exact file (and sometimes function name) in `src/` where the idea just explained is actually implemented in code, so you can go read the real, working implementation directly if you want to.

CAUTION / LIMITATION

This box is used whenever a method has a genuine, honestly-acknowledged weakness or a subtlety that is easy to get wrong — this project treats "what could go wrong with this" as important as "how does this work."

A map of the whole project, in one paragraph

The project builds three increasingly realistic mathematical descriptions of how a stock (or Bitcoin) price moves, "teaches" all three to match the real historical behaviour of SPY (a calm U.S. equity index fund) and BTC-USD (a volatile cryptocurrency), then simulates a market maker who has sold a call option and must hedge it by trading the underlying asset every day for three months. It does this thousands of times over, under many different assumptions about the "hedge model" the market maker believes in, and measures how bad the worst outcomes are using a statistic called **Conditional Value-at-Risk (CVaR)**. On top of that core experiment, the project adds an entire practitioner's toolkit around it: confidence intervals on every number, a separate theory for extreme/rare outcomes, American-style (early-exercisable) options, faster numerical pricing methods, day-by-day profit-and-loss explanations, deliberate crash-scenario stress tests, and regulatory-style model validation — and wraps all of it in a 16-tab interactive dashboard. This guide covers every one of those pieces, in the order a newcomer would most naturally want to learn them.

PART 2

Foundations — Options, Hedging, and Why Any of This Matters

Before any formula, we need to agree on what an option is, who a market maker is, what "hedging" means, and why real markets are harder to model than textbooks suggest.

2.1 What Is an Option, Really?

Imagine a piece of paper (today, just an electronic entry) that gives you the **right, but not the obligation**, to buy one share of a stock at a price fixed today, at some point in the future. That piece of paper is called a **call option**. If it instead gives you the right to *sell* a share at a fixed price, it is called a **put option**.

PLAIN ENGLISH

Think of a call option like a coupon that says "you may buy one share of SPY for exactly \$683 any time in the next 3 months, if you want to." If SPY's actual price rises to \$700, your coupon is valuable — you can use it to buy at \$683 and immediately be \$17 richer. If SPY falls to \$650 instead, you simply throw the coupon away; you were never *obligated* to buy, so you lose nothing beyond what you originally paid for the coupon itself.

Four pieces of vocabulary describe every option completely, and they recur on every page of this guide:

Term	Meaning
Underlying (S)	The actual asset the option is written on — in this project, either SPY (an equity index fund) or BTC-USD (Bitcoin priced in US dollars).
Strike (K)	The fixed price written into the option — the price you're allowed to buy (call) or sell (put) at.
Maturity / Expiry (T)	The date the option stops existing. This project mostly studies 3-month (0.25-year) options.
Premium	The price you pay <i>today</i> to own the option — this is the number every pricing model in this project ultimately computes.

At the maturity date, a call option is worth exactly $\max(S_T - K, 0)$ — the underlying's final price minus the strike, or zero if that would be negative (you'd never "exercise" an option that lets you buy above the current market price). This "max(·, 0)" shape is called the option's **payoff**, and every pricing model in this project is, at its core, a different mathematical way of answering: *"given everything we know about how the underlying moves between now and maturity, what is the fair price today for a payment of $\max(S_T - K, 0)$ that will only be received later?"*

2.2 What Does a Market Maker Do?

A **market maker** is a firm (or trading desk) that stands ready to both buy and sell an asset — in this case, options — at almost any moment, profiting from the small difference between its buy price and sell price (the "bid-ask spread"), rather than from betting on which direction the market will move. If a client wants to buy a call option, the market maker sells it to them.

PLAIN ENGLISH

Picture a market maker as a shopkeeper for options instead of groceries. A customer walks in and buys a "3-month SPY call option," and the shopkeeper (market maker) hands it over and pockets the premium. But unlike a grocer who has already paid a wholesaler for the item they just sold, the market maker has just taken on a genuine financial **risk**: if SPY rallies hard over the next three months, they may owe the customer a very large payout at expiry, funded only by the (much smaller) premium they collected upfront.

This is the central tension the whole project is about: the market maker has received a fixed, known amount of cash today (the premium) in exchange for an uncertain, possibly very large future liability (the payoff at maturity). They cannot just do nothing and hope for the best — they must actively manage that risk. That active risk management is called **hedging**.

2.3 What Does It Mean to "Hedge"?

The key insight, first formalized by Fischer Black and Myron Scholes in 1973 (and by Robert Merton in the same year), is that a market maker can approximately cancel out their option risk by holding a carefully chosen, constantly-adjusted quantity of the underlying asset itself alongside the option. This quantity is called **Delta**,

written Δ , and it represents "how many shares of the underlying you need to hold, per option sold, so that a \$1 move in the stock price causes an (approximately) offsetting $\$1 \times \Delta$ move in your stock position, cancelling out the option's own sensitivity to that same \$1 move."

PLAIN ENGLISH

Suppose the option you sold behaves such that a \$1 rise in SPY makes the option you owe worth \$0.50 more (this "\$0.50 per \$1" sensitivity is exactly what Delta measures). If you also personally own 0.50 shares of SPY for every option you sold, then that same \$1 rise makes your shares worth \$0.50 more too. Your gain on the shares offsets your loss on the option (you owe more because you're short the option), leaving your *total* wealth roughly unchanged no matter which way SPY moves next. That is "Delta hedging" in one sentence: hold Δ shares against every option you're short, so small price moves wash out.

The word "approximately" is doing a lot of work in that description, for two reasons this project spends enormous effort quantifying:

1. **Delta itself changes as the stock price moves** (this "Delta-of-Delta" sensitivity is called **Gamma**, Γ , explained fully in Part 3), so a hedge that was perfect a moment ago drifts out of alignment as time passes and prices move, requiring the market maker to keep **trading more shares to "rebalance" the hedge** — and every trade costs money (a transaction cost).
2. **Delta hedging only protects against small, smooth price moves.** If the price suddenly *jumps* by a large amount between one rebalancing and the next (an earnings shock, a crash, a flash crash), the hedge that was correct an instant before the jump is now badly wrong, and the market maker suffers a real loss that no amount of "hedging more often" can fully prevent if the jump happens between two rebalancing dates. This is precisely the "hidden tail risk" this entire project sets out to measure.

2.4 Volatility, Drift, and Randomness

To price an option, we first need a mathematical description of how the underlying price itself might evolve. The simplest possible honest description says: over any short interval of time, the price's percentage change is a combination of (a) a predictable average trend, called **drift**, and (b) a random, unpredictable shock, whose typical size is controlled by a number called **volatility**, written σ (the Greek letter "sigma").

PLAIN ENGLISH

Volatility is just "how much does this asset's price typically wiggle around per year?", expressed as a percentage. SPY's volatility in this project is calibrated at about 15.8% per year — a calm, typical equity index. Bitcoin's is about 45.6% per year — nearly three times as wiggly, which matches the common experience that crypto prices swing much more wildly than a diversified stock index fund. A HIGHER volatility means a WIDER range of plausible future prices, which (all else equal) makes options on that asset more valuable — more volatility means more chance of a large favourable move before expiry.

The standard mathematical model for "drift plus random shocks" is called **Geometric Brownian Motion (GBM)**, and it is the engine underneath the Black-Scholes model (Part 3.1). Two extensions of GBM are the

heart of this project's more advanced models: letting the volatility σ itself be random and changing over time (the Heston model, Part 3.3), and letting the price occasionally make a sudden, discontinuous jump rather than always moving smoothly (the Merton model, Part 3.5).

2.5 Why "Jumps" Are the Whole Point of This Project

Geometric Brownian Motion assumes that if you zoom in on a price chart far enough, it always looks smooth and continuous — the price might move a lot over a year, but over one particular *second*, the move is always tiny. Real markets do not obey this rule perfectly. Earnings surprises, geopolitical shocks, flash crashes, and (for crypto) exchange failures all cause prices to move a large amount **almost instantaneously** — a "jump" rather than a smooth drift-and-wiggle.

WHY THIS MATTERS (THE PROJECT'S CENTRAL QUESTION)

A market maker who computes their hedge Delta using a model that CANNOT see jumps (Black-Scholes) will have their hedge blindsided every time a real jump occurs — their carefully rebalanced position was correct for the smooth world their model imagines, but wrong for the jumpy world that actually happened. A market maker who instead computes Delta using a model that DOES account for jump risk (Merton) will have priced and hedged the option slightly differently from day one, in a way that turns out to leave them meaningfully safer when a real jump occurs. **Quantifying exactly how much safer, in dollars and in a rigorous statistical sense, is what this entire project does.**

The rest of this guide builds up, piece by piece, everything needed to make that comparison rigorous: three pricing models of increasing realism (Part 3), a way to "teach" each model what a real asset's jumps and volatility actually look like from historical data (Part 4), a simulator that plays out thousands of alternate futures and has a market maker hedge through each one (Part 5), a way to measure "how bad are the worst outcomes" (Part 6), and an entire apparatus of statistical rigor, alternative risk theories, and numerical methods layered on top (Parts 7 through 11) to make sure the final answer is trustworthy rather than a fluke of one particular simulation.

PART 3

The Three Pricing Models

Black-Scholes, Heston, and Merton — three increasingly realistic mathematical universes an option can be priced in, each formula broken down piece by piece.

Every option pricing model answers the same question — "what is a fair price today for a payment of $\max(S_T - K, 0)$ at time T ?" — but each makes a different assumption about how the underlying price S actually moves between now and T . This project implements three such models, each one relaxing an assumption of the previous one, and validates each against at least one independent method before trusting it (a discipline explained more in Part 3.6).

3.1 Black-Scholes-Merton: The Baseline Model

Black-Scholes assumes the underlying follows Geometric Brownian Motion (Part 2.4) with a CONSTANT volatility σ and no jumps — the simplest honest starting point, and the model every other model in this project is checked against in the appropriate limiting case.

FORMULA — THE BLACK-SCHOLES CALL PRICE

$$C = S \cdot e^{-qT} \cdot N(d_1) - K \cdot e^{-rT} \cdot N(d_2)$$

$$d_1 = [\ln(S/K) + (r - q + \sigma^2/2) \cdot T] / (\sigma \cdot \sqrt{T}) \quad d_2 = d_1 - \sigma \cdot \sqrt{T}$$

S	Current price of the underlying (e.g., SPY today).
K	Strike price written into the option contract.
T	Time remaining to maturity, measured in years (a 3-month option has $T = 0.25$).
r	The risk-free interest rate (what you could earn with zero risk, e.g., from a government bond) — used to "discount" a future payment back to today's dollars.
q	The underlying's dividend yield (0 for non-dividend-paying assets like BTC-USD; used later in Part 9 to show WHY early exercise of an American call can become optimal).
σ	Volatility — the annualized "typical wiggle size" defined in Part 2.4.
$N(x)$	The cumulative normal distribution function : literally, "the probability that a random draw from the standard bell curve is less than x." $N(0) = 0.50$ (a coin-flip chance of being below the average); $N(2) \approx 0.977$ (a random bell-curve draw is very likely below 2 standard deviations above the mean).
$\ln$	The natural logarithm — here, $\ln(S/K)$ measures how far in percentage terms the current price is from the strike (positive if $S > K$, negative if $S < K$).

PLAIN ENGLISH

$N(d_2)$ turns out to equal (under this model) the actual probability that the option will be exercised at all (that S_T ends up above K). $N(d_1)$ is a closely related, slightly different probability that shows up because we also have to account for HOW FAR above K the price ends up, not just whether it does. The whole formula can be read as: *"the current price of the underlying, weighted by one probability-like quantity, minus the discounted strike, weighted by the true probability of exercise."* Both terms are discounted/probability-weighted versions of "what you'd pay/receive if you knew for certain the option would be exercised."

WORKED EXAMPLE

Take Hull's classic textbook numbers: $S = \$42$, $K = \$40$, $T = 0.5$ years, $r = 10\%$, $\sigma = 20\%$ (this exact example is used as this project's own sanity check in `black_scholes.py`). Plugging in: $d_1 \approx 0.7693$, $d_2 \approx 0.6278$, $N(d_1) \approx 0.7791$, $N(d_2) \approx 0.7349$. The call price comes out to $C = 42(0.7791) - 40 \cdot e^{-0.05}(0.7349) \approx \mathbf{\$4.76}$ — matching the textbook to the penny, and matching this project's own automated test in `tests/test_core_functions.py`.

WHERE THIS LIVES IN THE PROJECT

`src/black_scholes.py`, function `bs_price()`. Validated against Hull's textbook benchmark and against put-call parity (a model-free arithmetic identity: Call – Put must always equal $S - K \cdot e^{-rT}$, a relationship that holds regardless of which pricing model you use, because it follows purely from the payoffs, not from any assumption about how S moves).

3.2 The Greeks: Measuring Sensitivity

The "Greeks" are a family of sensitivities — each one answers "if *this one thing* changes by a tiny amount, how much does the option's price change?" They are called Greeks because most (though not all) are named after Greek letters. They matter because hedging (Part 2.3) is entirely about knowing and managing these sensitivities.

Greek	Measures sensitivity of price to...	Plain-English meaning
Delta (Δ)	The underlying price S	How many shares of the underlying to hold to hedge one option (Part 2.3).
Gamma (Γ)	Delta itself, as S changes	How fast your hedge goes stale as the price moves — high Gamma means you need to rebalance more often.
Vega	Volatility σ	How much the option's price changes if the market's expectation of future wiggleness changes — not hedged at all by trading the underlying (Part 5.4 explains why this matters).
Theta (Θ)	Time itself (as T shrinks)	"Time decay" — how much value an option loses each day purely from having one fewer day left to be profitable, holding everything else fixed.
Rho (ρ)	The interest rate r	How much the price changes if interest rates move — a minor effect for short-dated options, included here for completeness.

FORMULA — DELTA AND GAMMA FOR A CALL (BLACK-SCHOLES)

$$\Delta = e^{-qT} \cdot N(d_1) \quad \Gamma = e^{-qT} \cdot N'(d_1) / (S \cdot \sigma \cdot \sqrt{T})$$

$N'(x)$ The standard normal *probability density* (the height of the bell curve at x) — not to be confused with $N(x)$, the area/probability under the curve up to x .

WORKED EXAMPLE

In the Hull example above, Delta ≈ 0.779 : for every option sold, the market maker holds about 0.78 shares of the underlying. If SPY were the underlying and the market maker sold 100 call options (a contract for 100 shares each, so 10,000 "option-shares" of exposure), they would hold approximately 7,790 actual shares of SPY to be Delta-neutral at that instant.

WHERE THIS LIVES IN THE PROJECT

`src/black_scholes.py`, function `bs_greeks()`, returning a dictionary with all five Greeks at once. For the Heston and Merton models (below), most Greeks have no clean closed-form formula, so they are instead computed by **finite differences**: nudge one input (say S) up by a tiny amount, nudge it down by the same tiny amount, price the option both times, and divide the difference in price by the (tiny) difference in the input — literally re-deriving the definition of a derivative/slope numerically, without needing an exact formula.

3.3 Heston: When Volatility Itself Is Random

Black-Scholes assumes σ is one fixed number for the whole life of the option. In reality, volatility itself rises and falls over time — it tends to spike during crises and settle back down during calm periods, a phenomenon called **mean reversion**. Steven Heston's 1993 model captures this by making the *variance* (volatility squared, written v_t) its own separate random process, correlated with the price process.

FORMULA — THE HESTON MODEL'S TWO COUPLED EQUATIONS

$$dS_t = r \cdot S_t \cdot dt + \sqrt{v_t} \cdot S_t \cdot dW_t^S$$

$$dv_t = \kappa(\theta - v_t) \cdot dt + \xi \sqrt{v_t} \cdot dW_t^v$$

$$\text{corr}(dW_t^S, dW_t^v) = \rho$$

v_t	The instantaneous variance at time t (so $\sqrt{v_t}$ is the instantaneous volatility — it now CHANGES over time instead of being one fixed σ).
κ	("kappa") Mean-reversion speed : how quickly variance gets pulled back toward its long-run average after a shock. A bigger κ means volatility spikes fade faster.
θ	("theta", reused here as a Heston parameter, not the Theta Greek) The long-run average variance that v_t keeps drifting back toward.
ξ	("xi") Vol-of-vol : how random and jumpy the VARIANCE process itself is. If $\xi=0$, variance would move in a perfectly predictable straight line toward θ — realistically, $\xi>0$ always.
ρ	("rho") The correlation between price shocks and variance shocks. Empirically almost always negative (the "leverage effect"): when prices fall, fear rises, and volatility tends to spike at the same time.
dW^S, dW^v	Two random shocks (technically, increments of two correlated "Brownian motions") — the mathematical device that injects randomness into each equation, analogous to the single random shock in Geometric Brownian Motion but now there are two, correlated with each other by ρ .

PLAIN ENGLISH

Heston has two "moving parts" instead of one: the price itself, and the **CURRENT LEVEL** of volatility, which is allowed to wander around, spike, and settle back down on its own. This is a much better match for reality, where a quiet market can become turbulent (variance shoots up) and then calm back down over subsequent weeks — something Black-Scholes' single fixed σ can never represent.

Why Heston has no simple price formula, and what a "characteristic function" is

Black-Scholes has a clean formula because the underlying's final log-price follows an ordinary bell curve (Normal distribution), whose probabilities $N(x)$ are well understood and tabulated. Under Heston, the final log-price's distribution is much more complicated and has no simple bell-curve shape — but it turns out to have a

known **characteristic function**: a mathematical fingerprint of a probability distribution, built from a Fourier transform, that fully describes the distribution even when its ordinary probability formula is intractable.

PLAIN ENGLISH

Think of the characteristic function as a distribution's "barcode": you can't necessarily read off "the probability of X being below 5" just by looking at the barcode directly, but there is a well-known mathematical scanner (a formula called the **Gil-Pelaez inversion**) that takes the barcode as input and computes any probability you want as output, via an integral (a continuous sum). This project's `heston.py` computes that "barcode" in closed form (a genuinely nasty-looking but entirely mechanical formula involving complex numbers), then numerically evaluates the "scanner" integral using `scipy.integrate.quad` to get an actual option price out the other end.

CAUTION / LIMITATION

That "scanner" integral (`scipy.integrate.quad`) only works on ONE price at a time — it fundamentally cannot be handed a whole array of thousands of prices at once the way Black-Scholes' simple formula can. This made the original Heston pricer roughly $1,000\times$ slower than Black-Scholes/Merton for large-scale simulation, and is the exact limitation Part 3.4 (the COS method) was built to remove.

WHERE THIS LIVES IN THE PROJECT

`src/heston.py : _heston_char_func()` computes the "barcode" (using the numerically-stable "Little Trap" formulation of Albrecher et al., 2007, chosen specifically because the textbook 1993 version of this formula has known numerical blow-up issues for long maturities). `heston_price()` runs the "scanner" integral. Validated against independent Monte Carlo simulation (Part 10 explains what that is) of the same two equations above, and checked to match Black-Scholes exactly in the limiting case $\xi \rightarrow 0$ (no vol-of-vol means variance is effectively constant, which is exactly Black-Scholes' assumption).

3.4 The COS Method: Making Heston Fast

The Fourier-Cosine (COS) method, published by Fang & Oosterlee in 2008, is a second, mathematically different way to use that same "barcode" (characteristic function) to get an option price — one that turns out to be both highly accurate AND fully vectorizable (able to price many spot prices at once, the way Black-Scholes can).

PLAIN ENGLISH

Instead of computing one exact "scanner" integral per price (slow, one at a time), the COS method notices that ANY reasonably smooth function (including the payoff of an option) can be approximated extremely accurately as a sum of a few hundred cosine waves of different frequencies — the exact same idea used in audio compression, where a sound wave is approximated as a sum of pure tones. Once you know how much of each cosine "frequency" is present in both the probability distribution (from the characteristic function) and the payoff, pricing an option becomes a simple, finite weighted sum instead of an infinite integral — and because the expensive part (evaluating the characteristic function at each of a few hundred frequencies) does not depend on which specific spot price you plug in afterward, it can be computed ONCE and reused for thousands of different spot prices simultaneously using ordinary array arithmetic.

FORMULA — THE COS PRICING SUM

$$\text{Price} \approx e^{-rT} \cdot \sum_{k=0}^{N-1} \text{Re}[\varphi(k\pi/(b-a)) \cdot e^{-ik\pi a/(b-a)}] \cdot V_k$$

$\varphi(\cdot)$	The characteristic function ("barcode") from Part 3.3, evaluated at a handful of specific frequencies instead of integrated continuously.
V_k	The "cosine content" of the option's payoff at frequency k — a closed-form expression, since the payoff (a simple kinked straight-line shape) has an exactly-known cosine decomposition.
$[a, b]$	A truncation range wide enough to contain essentially all of the probability mass of the underlying's future log-price — chosen generously (about 12 standard deviations wide in this project) so the approximation is, in practice, indistinguishable from exact.
N	Number of cosine terms used (256 in this project) — more terms means more accuracy, with rapidly diminishing need for more once N is in the low hundreds.
$\text{Re}[\cdot]$	"Real part of" — the characteristic function briefly involves complex numbers internally, but the final price is always a plain real number.

THE PAYOFF OF THIS UPGRADE

Cross-validated against the original "scanner integral" pricer across 20 different strike/maturity combinations (agreement to within 0.03%), confirmed to still satisfy put-call parity exactly, and benchmarked at roughly **180 times faster** when pricing 500 different spot prices at once. This upgrade is what let the project's Heston-related studies run at the SAME 8,000-path, daily-rebalancing scale as the other two models, instead of a 150-path approximation to it (see Parts 12 and 13 for where this shows up).

WHERE THIS LIVES IN THE PROJECT

`src/heston.py`, functions `heston_price_cos()` (one price at a time) and `heston_price_cos_batch()` (many spot prices at once, the version actually used for large-scale simulation). The batch version exploits a mathematical fact called **scale invariance**: Heston's equations only care about the underlying's PERCENTAGE moves, not its absolute price level, so the expensive "evaluate the barcode at 256 frequencies" step can be done once using a reference price of \$1 and reused, unchanged, for every one of thousands of different actual spot prices in a batch.

3.5 Merton: When Prices Can Jump

Robert Merton's 1976 model takes a different extension of Black-Scholes: instead of letting volatility itself be random (Heston's approach), it keeps volatility fixed but adds the possibility of sudden, discontinuous **jumps** in the price — directly modelling the crash risk described in Part 2.5.

FORMULA — THE MERTON JUMP-DIFFUSION EQUATION

$$dS_t/S_t = (r - \lambda k - \sigma^2/2) \cdot dt + \sigma \cdot dW_t + dJ_t$$

λ ("lambda") The **jump intensity**: the expected NUMBER of jumps per year. This project calibrates $\lambda \approx 3.67/\text{year}$ for SPY and $\approx 4.26/\text{year}$ for BTC-USD — i.e., a jump roughly every 3-4 months on average, for both.

J_t A **compound Poisson process**: jumps arrive at random times (governed by λ), and each jump's SIZE is itself random, drawn from its own distribution (below).

k The expected PERCENTAGE size of a jump, $k = E[e^{\text{jump}} - 1]$. Subtracting λk from the drift is a bookkeeping correction (the "compensator") that keeps the AVERAGE growth rate of the stock equal to r even after jumps are added — without it, adding jumps would silently bias the average return.

Each individual jump's log-size is drawn from its own (separate) bell curve:

$$\ln(1 + \text{Jump}) \sim \text{Normal}(\mu_J, \sigma_J^2)$$

μ_J The average log-size of a jump — calibrated strongly NEGATIVE for both assets (about -1.1% for SPY, -3.4% for BTC-USD), matching the well-documented tendency for crashes to be larger and more sudden than rallies.

σ_J How much jump sizes themselves vary from one jump to the next.

PLAIN ENGLISH

Between jumps, Merton's price moves exactly like ordinary Black-Scholes (smooth drift plus wiggle). But every so often — on average λ times per year — a jump occurs: the price is suddenly multiplied by a random factor, usually a modest downward jolt, occasionally a larger one, representing a crash, an earnings shock, or (for crypto) an exchange failure. A market maker hedging with Black-Scholes Delta has no way to anticipate or price this possibility at all; a market maker using Merton Delta has, from day one, priced in a small insurance premium against exactly this risk, and adjusts their hedge accordingly.

How Merton is actually priced: a weighted sum of Black-Scholes prices

Remarkably, Merton found a quasi-closed-form solution: the price is simply a weighted AVERAGE of ordinary Black-Scholes prices, one for every possible NUMBER of jumps (0, 1, 2, 3, ...) that could occur before maturity, weighted by the probability of exactly that many jumps occurring (a Poisson probability).

FORMULA — THE MERTON PRICE

$$\text{Price} = \sum_{n=0}^{\infty} [e^{-\lambda'T} (\lambda'T)^n / n!] \cdot \text{BS}_{\text{price}}(S, K, T, r_n, \sigma_n)$$

n	A specific, fixed number of jumps (0, 1, 2, ...) — each term in the sum asks "IF exactly n jumps happen, what is the Black-Scholes price, and how likely is exactly n jumps?"
$e^{-\lambda'T} (\lambda'T)^n / n!$	The Poisson probability of exactly n jumps occurring in the T years remaining — a standard formula from probability theory for "rare events happening a specific number of times."
σ_n	An adjusted volatility for the n -jump scenario: $\sqrt{(\sigma^2 + n \cdot \sigma_J^2) / T}$ — more jumps means more total randomness, which this formula folds into an "effective" volatility for that scenario.

In practice the infinite sum is cut off after about 50 terms, since the Poisson probability of, say, 30+ jumps in a 3-month option is astronomically small and contributes nothing measurable to the total.

WHERE THIS LIVES IN THE PROJECT

`src/merton.py`, function `merton_price()`, which literally loops over n and calls `black_scholes.bs_price()` inside the loop — a clean illustration of how a more advanced model can REUSE a simpler one's already-validated code rather than re-deriving everything from scratch. Validated against independent Monte Carlo simulation of the jump-diffusion equation above, and checked to reduce exactly to plain Black-Scholes as $\lambda \rightarrow 0$ (no jumps at all).

3.6 "Never Trust a Single Untested Pricer"

A theme repeated throughout this project: every pricing formula is checked against at least one INDEPENDENT method before being trusted for the main experiment. The three most common independent checks used are:

1. **Textbook / known-benchmark matching** — comparing against a published, widely-cited numeric example (used for Black-Scholes).
2. **Monte Carlo cross-validation** — simulating the SAME underlying equation thousands of times with computer-generated randomness and averaging the resulting payoffs (fully explained in Part 10), then checking the closed-form/semi-closed-form price agrees within statistical noise (used for Heston and Merton).
3. **Nested-model reduction** — checking that a more complex model, with its "extra" parameter set to an extreme value (no vol-of-vol, no jumps), collapses EXACTLY back onto the simpler model it was built to extend.

This discipline matters because a subtle coding error in a pricing formula can produce numbers that look plausible but are simply wrong — and since this project's entire conclusion rests on comparing dollar amounts of risk across models, an unvalidated pricer would quietly invalidate everything built on top of it.

PART 4

Calibration — Teaching the Models About the Real World

A model is just an empty formula until its parameters (σ , λ , κ , θ , ξ , ρ ...) are pinned down by real data. This part explains how, and why the specific choice made here matters.

Every parameter introduced in Part 3 — σ , λ , μ_J , σ_J for Merton; κ , θ , ξ , ρ , v_0 for Heston — is just a placeholder until it is given an actual number. **Calibration** is the process of choosing those numbers so the model is consistent with a particular real asset (SPY or BTC-USD, in this project).

4.1 Two Fundamentally Different Ways to Calibrate

There are two distinct, philosophically different sources of data you could use to calibrate an option-pricing model, and confusing them is one of the most common real mistakes in quantitative finance:

Approach	Uses...	Answers the question...
Historical (P-measure)	The asset's own past price history (what actually happened).	"What would a model consistent with how this asset has behaved <i>in the past</i> say?"
Implied (Q-measure)	Current, live prices of traded options on that asset (what the market is willing to pay <i>right now</i>).	"What is the market currently pricing risk at, regardless of what happened historically?"

PLAIN ENGLISH

Historical calibration is like predicting tomorrow's weather from a detailed record of the last eight years of weather in your city. Implied calibration is like reading today's insurance premiums for storm damage and reverse-engineering what the insurance company must currently believe the odds of a storm are. Both are legitimate ways to estimate risk, but they can disagree — insurance premiums might be elevated right now because of a recent scary news story, even if the long-run historical storm frequency hasn't actually changed. Mixing the two up (using a historical estimate but describing it as "what the market thinks") is a well-known and important mistake to avoid.

CAUTION / LIMITATION (EXPLICITLY ACKNOWLEDGED IN THIS PROJECT)

This project uses **historical (P-measure) calibration exclusively** for its main pipeline, because genuinely free, long-history options-chain data (needed for implied/Q-measure calibration) essentially does not exist for public download. This is a real, clearly weaker substitute for implied calibration, and is called out explicitly rather than glossed over. The dashboard's **Live Market Check** tab (Part 13.3) partially closes this gap by comparing the historical model's smile against SPY's actual, live, currently-traded options chain — a genuine, if narrower and non-historical, Q-measure comparison.

4.2 Calibrating Merton: Finding the Jumps in a Return Series

To calibrate Merton's four parameters (σ , λ , μ_J , σ_J) from a plain list of an asset's daily percentage returns, this project uses a simple, transparent **threshold-based** procedure:

1. **Flag "jump days."** Compute the standard deviation of ALL daily returns, then flag any day whose return is more than 3 standard deviations away from the average as a "jump."
2. **Estimate the jump intensity λ .** Count how many days were flagged, divide by the number of YEARS of data available. If 29 jumps were flagged over 7.9 years of SPY data, $\lambda = 29/7.9 \approx 3.67$ jumps per year.
3. **Estimate the jump-size distribution.** Take the AVERAGE and STANDARD DEVIATION of just the flagged jump-day returns — these become μ_J and σ_J directly.
4. **Estimate the diffusive volatility σ .** Use the variance of the REMAINING (non-jump) days for the ordinary Black-Scholes-style volatility, since the jump days' extra variance has already been accounted for separately.

WORKED EXAMPLE — ACTUAL CALIBRATED NUMBERS FROM THIS PROJECT

Asset	σ (diffusive vol)	λ (jumps/year)	μ_J	σ_J
SPY (calm equity)	15.8%	3.67	-1.08%	6.14%
BTC-USD (volatile crypto)	45.6%	4.26	-3.36%	13.76%

Two things jump out (no pun intended): Bitcoin's diffusive volatility is nearly $3\times$ SPY's even BEFORE considering jumps at all, and its jump-size distribution is both larger in magnitude and more negatively skewed — both consistent with the well-documented reality that crypto markets crash harder and more often than a diversified equity index.

WHERE THIS LIVES IN THE PROJECT

`src/calibration.py`, function `calibrate_merton()`. Validated with a "recovery test": simulate a KNOWN set of parameters using `merton_simulate_paths()`, calibrate on the simulated data, and confirm the recovered numbers land in a reasonable ballpark of the true, known values that generated the data in the first place — you cannot expect exact recovery from any single finite sample, so this is a sanity check, not a proof.

4.3 Calibrating Heston: Reading Volatility's Own History

Heston's five parameters are calibrated from the asset's **realized volatility** — a rolling, backward-looking measure of "how much did this asset actually wiggle over the last 21 trading days" (about one calendar month), computed directly from historical returns.

Parameter	How it is estimated
v_0	The most RECENT 21-day realized variance — "how volatile has this asset been lately?"
θ	The LONG-RUN AVERAGE realized variance across the entire history — "what's normal for this asset?"
κ	Estimated via a simple regression (see below) of day-to-day CHANGES in variance against how far variance currently is from θ .
ξ	The leftover, unexplained "noise" in that same regression — how much variance moves that ISN'T explained by simple mean-reversion.
ρ	The plain statistical correlation between daily returns and daily CHANGES in realized variance.

PLAIN ENGLISH — WHAT IS A "REGRESSION" DOING HERE?

A regression is simply a best-fit straight line through a scatter of points. Here, each point plots "how far today's variance is from the long-run average θ " (horizontal axis) against "how much variance actually changed from today to tomorrow" (vertical axis). If variance genuinely mean-reverts, this scatter should show a clear pattern: when variance is well above θ , it tends to fall the next day; when it's well below θ , it tends to rise. The STEEPNESS of the best-fit line through that scatter is exactly κ (how strongly and quickly variance gets pulled back), and how SCATTERED the points are around that line (i.e., how much of the day-to-day change ISN'T explained by the line) becomes ξ , the vol-of-vol.

WORKED EXAMPLE — ACTUAL CALIBRATED NUMBERS FROM THIS PROJECT

Asset	κ	θ
SPY (calm equity)	1.92	0.0387 ($\sqrt{\theta} \approx 19.7\%$ long-run vol)
BTC-USD (volatile crypto)	6.87	0.2921 ($\sqrt{\theta} \approx 54.0\%$ long-run vol)

BTC-USD's much higher κ means its volatility spikes fade back to normal roughly $3.6\times$ faster than SPY's — consistent with crypto markets being prone to sharp, short-lived volatility spikes that resolve quickly, rather than the more gradual volatility regimes typical of equity indices.

WHERE THIS LIVES IN THE PROJECT

`src/calibration.py`, function `calibrate_heston()`. Also validated with a recovery test on simulated data with known, pre-chosen parameters (via `heston_simulate_paths()`).

PART 5

The Hedging Engine — Simulating a Market Maker's Life

Given a pricing model, how do we actually simulate a market maker hedging day by day, and keep honest track of every dollar?

5.1 The Replication Argument and the Cash Account

The simulator in this project (`hedging_engine.py`) plays out, one day at a time, exactly what a real market maker would do: receive a premium, buy some shares, and then keep adjusting that share position every day until the option expires. To track whether this strategy actually worked, it keeps a simple **cash account** — a running ledger of every dollar in and out.

FORMULA — THE CASH ACCOUNT, DAY BY DAY

Day 0 (option sold):

$$\text{cash}_0 = V_0 - \Delta_0 \cdot S_0$$

Every later day t (rebalancing):

$$\text{cash}_t = \text{cash}_{t-1} \cdot e^{r \cdot dt} - (\Delta_t - \Delta_{t-1}) \cdot S_t - \text{cost}_t$$

At maturity T (settlement):

$$\text{Hedging P\&L} = \text{cash}_T + \Delta_T \cdot S_T - \text{payoff}$$

V_0	The option's premium, received on day 0 in exchange for selling (going "short") the option.
$\Delta_t \cdot S_t$	The cost of buying Δ_t shares at today's price — funded out of the cash account.
$e^{r \cdot dt}$	Cash sitting in the account between rebalances earns the risk-free interest rate r — it doesn't just sit idle.
cost_t	The transaction cost paid to trade shares today — modelled as a small percentage (5 basis points = 0.05%) of the dollar value traded, a realistic proxy for a bid-ask spread.
payoff	What the market maker OWES the option buyer at expiry: $\max(S_T - K, 0)$ for a call.

PLAIN ENGLISH

Every day, three things happen: (1) whatever cash is sitting in the account earns a little interest, (2) the market maker checks whether their current share holding still matches the option's current Delta, and if not, buys or sells just enough shares to fix that (paying a small trading fee either way), and (3) at the very end, the market maker sells whatever shares they're still holding, pays out the option's actual payoff to the buyer, and whatever cash is left over (positive or negative) IS the hedging profit or loss for that simulated path. If Delta hedging worked perfectly, this final number would be exactly zero every time — in practice, it is a small random number that averages close to zero under a MATCHING model, and a much more variable, sometimes very negative number when the hedge model doesn't match how the world actually moved (Part 6 explains how this is measured rigorously).

WHERE THIS LIVES IN THE PROJECT

`src/hedging_engine.py`, function `simulate_delta_hedge()` (one price path at a time) and `simulate_delta_hedge_batch()` (an entire array of thousands of price paths simultaneously, for speed — validated to produce EXACTLY the same numbers as the one-path-at-a-time version, just about 1,000× faster, since it processes every path for one day at once instead of every day for one path at once).

5.2 Fixed-Frequency Rebalancing and the Cost/Risk Tradeoff

How OFTEN should the market maker check and adjust their Delta? Rebalancing every single day gives the tightest possible hedge (Delta almost never drifts far from correct), but generates a transaction cost every single day. Rebalancing only once a week is cheaper, but lets Delta drift further out of alignment between adjustments, increasing risk.

WHAT THIS PROJECT FOUND

Sweeping the rebalancing frequency from daily out to monthly, holding everything else fixed, confirms the textbook tradeoff directly: CVaR of hedging loss (Part 6) rises monotonically as rebalancing becomes less frequent — +15.5% from daily to monthly for SPY, and an even steeper +52.6% for BTC-USD, whose higher Gamma (faster-changing Delta) makes it much more sensitive to going a long time between adjustments.

5.3 Band ("No-Transaction Region") Hedging: A Smarter Schedule

Real trading desks rarely rebalance on a rigid calendar. A more realistic practical rule — going back to Whalley & Wilmott (1993) — is to MONITOR Delta continuously, but only actually TRADE once it has drifted more than some tolerance "band" away from the currently-held position.

PLAIN ENGLISH

Imagine a thermostat with a target temperature and a small tolerance window (say, $\pm 1^\circ\text{F}$) rather than one that runs the heater continuously to hit an exact number. The furnace only turns on once the room drifts outside that window, saving energy without letting the room get uncomfortably cold. Band hedging applies the same idea to Delta: don't trade every single day regardless of need — trade only when your current hedge has drifted far enough from the model's current Delta to actually be worth the transaction cost of fixing.

WHAT THIS PROJECT FOUND

For SPY, widening the tolerance band MONOTONICALLY reduced both CVaR and the average transaction cost paid — band hedging is not simply "less trading equals more risk." For BTC-USD, CVaR initially FELL as the band widened, reached a minimum around a band width of 0.05, and then rose again for wider bands still — a genuine cost/risk "sweet spot," not a trivial always-one-direction tradeoff.

WHERE THIS LIVES IN THE PROJECT

`src/hedging_engine.py`, functions `simulate_delta_hedge_band()` and its vectorized counterpart `simulate_delta_hedge_band_batch()`. Doubly validated: bit-identical to its own per-path version, AND proven to reduce EXACTLY to ordinary daily rebalancing when the tolerance band is set to zero (a difference of precisely 0.00, confirming it is a strict generalization rather than a separate, potentially-inconsistent implementation).

5.4 Delta-Vega Hedging: Adding a Second Instrument

Every hedge described so far trades ONLY the underlying asset. But recall from Part 3.2 that the underlying has zero Vega — it has no direct sensitivity to volatility itself changing. This means a Delta-only hedge can NEVER

protect against volatility risk, no matter how sophisticated the pricing model behind it is — a real limitation for the Heston model specifically, whose entire distinguishing feature IS random volatility.

The fix used in this project is to add a SECOND option (a different strike, same expiry) to the hedge portfolio, sized specifically to cancel out net Vega, with the underlying then used (as before) to mop up whatever Delta is left over.

FORMULA — THE TWO-INSTRUMENT HEDGE RATIOS

$$n_{\text{instrument}} = \text{Vega}_{\text{target}} / \text{Vega}_{\text{instrument}}$$

$$n_{\text{stock}} = \Delta_{\text{target}} - n_{\text{instrument}} \cdot \Delta_{\text{instrument}}$$

target	The option the market maker originally sold (is "short").
instrument	The second, different-strike option now added purely as a hedging tool.
$n_{\text{instrument}}$	How many units of the second option to hold — chosen so that $n_{\text{instrument}} \cdot \text{Vega}_{\text{instrument}}$ exactly cancels $\text{Vega}_{\text{target}}$, leaving zero NET Vega.
n_{stock}	How many shares of the underlying to hold — chosen AFTER the Vega hedge is set, to clean up whatever Delta the instrument itself introduced (since the instrument option has its own Delta too, not just Vega).

WHAT THIS PROJECT FOUND — AN HONEST, NUANCED RESULT

Tested under a true market with genuine stochastic volatility (each asset's own calibrated Heston process), Delta-Vega hedging cut CVaR by roughly a quarter for BTC-USD — a large, real improvement, because BTC's calibrated vol-of-vol ($\xi \approx 0.33$) is substantial. For SPY, whose calibrated vol-of-vol is much smaller ($\xi \approx 0.09$), adding a Vega hedge actually made CVaR slightly WORSE: the second instrument's own trading costs and noise were not repaid by protection against a volatility-of-volatility risk that barely exists for that asset. This is reported honestly rather than only showing the flattering case — it reinforces the project's broader theme that hedge sophistication must match the SPECIFIC risk actually present, not be added on principle.

CAUTION / LIMITATION — A GENUINE NUMERICAL SUBTLETY, FOUND AND FIXED

Every option's Vega shrinks to zero as expiry approaches, and a fixed-strike instrument that has drifted away from the money loses its Vega FASTER than an at-the-money target does — so the ratio $n_{\text{instrument}}$ can compound to an unreasonable size in the final days before expiry, well before either Vega is a literal zero. This was caught during development as an unusually heavy-tailed simulated profit/loss distribution, traced to exactly this cause, and fixed by capping the instrument quantity at a fixed multiple of its well-behaved, full-maturity value — approximating how a real trading desk would roll into a fresher hedging instrument rather than lever into an increasingly unreliable one.

WHERE THIS LIVES IN THE PROJECT

`src/hedging_engine.py` , function `simulate_delta_vega_hedge_batch()` , using a new `_batch_vega()` helper that computes Vega for all three hedge models (including Merton, which previously had no Vega calculation at all).

Measuring Risk — VaR and CVaR From Scratch

How do you turn "thousands of simulated hedging outcomes" into a single, honest number describing how bad things can get?

6.1 From Many Simulated Paths to a Loss Distribution

Running the hedging engine from Part 5 on ONE simulated price path gives ONE number: the hedging profit or loss for that particular pretend future. That single number tells you almost nothing about risk — you need to know the whole RANGE of things that could plausibly happen. This project's core experiment simulates 8,000 independent, alternate 3-month futures for each asset (each one a different random draw from the calibrated Merton jump-diffusion process), hedges through every single one, and collects all 8,000 resulting profit/loss numbers into a **distribution** — essentially, a very detailed histogram of "here is every plausible outcome, and how often each range of outcomes occurred."

For risk-measurement purposes, this project works with **losses** rather than profits: $\text{loss} = -(\text{hedging P\&L})$, so that a POSITIVE loss means the market maker lost money, and a bigger positive number is a worse outcome — a simple sign convention used consistently everywhere in the project.

6.2 Value-at-Risk (VaR): "How Bad, at the q% Worst Level?"

FORMULA — HISTORICAL VAR

$\text{VaR}_q = \text{the } (1-q) \text{ quantile of the loss distribution}$

q	The tail probability, e.g. $q=0.05$ for "95% VaR" (looking at the worst 5% of outcomes) or $q=0.01$ for "99% VaR" (the worst 1%).
quantile	Sort ALL the simulated losses from smallest to biggest. The $(1-q)$ quantile is the value that $(1-q)$ of the fraction of outcomes fall below — equivalently, exactly q of the fraction of outcomes are WORSE than this number.

WORKED EXAMPLE — VAR BY HAND ON 20 NUMBERS

Suppose (for illustration only — the real project uses 8,000 paths, not 20) a simulation produced these 20 sorted losses (in dollars), worst first:

95, 80, 62, 55, 48, 40, 35, 30, 28, 22, 18, 15, 12, 9, 5, 2, -1, -4, -8, -15

For 95% VaR ($q = 0.05$), we want the loss level that only the worst 5% of outcomes exceed. With 20 numbers, 5% is exactly 1 outcome. So $\text{VaR}_{5\%}$ is the SECOND-worst number in the sorted list (the boundary just past the single worst outcome): **$\text{VaR}_{5\%} = 80$** . Interpretation: "there is only a 5% chance of losing more than \$80."

CAUTION / LIMITATION OF VAR

VaR tells you the THRESHOLD the worst outcomes cross, but says NOTHING about how bad things get once you're past that threshold. In the example above, VaR is \$80 whether the single worst outcome was \$81 or \$10,000,000 — VaR is completely blind to just how catastrophic the tail actually is. This single weakness is the entire motivation for CVaR, below.

6.3 Conditional Value-at-Risk (CVaR): "How Bad, ON AVERAGE, Beyond That Point?"

FORMULA — HISTORICAL CVAR

$\text{CVaR}_q = \text{the AVERAGE of the worst } q\text{-fraction of all outcomes}$

k	The number of "worst" outcomes to average, $k = \text{floor}(q \times \text{total number of outcomes})$. In code: sort the losses worst-first, take the top k of them, and average just those k numbers (see the worked example below).
---	---

WORKED EXAMPLE — CVAR BY HAND, SAME 20 NUMBERS

Using $q = 0.05$ again on the same 20 sorted losses, $k = \text{floor}(0.05 \times 20) = 1$ — so $\text{CVaR}_{5\%}$ is simply the average of the single worst outcome: **$\text{CVaR}_{5\%} = 95$** (in this tiny 20-number example, CVaR at this q -level happens to just equal the single worst loss, since only one outcome falls in the worst 5%). With a bigger sample — say 100 outcomes and $q=0.05$ — CVaR would average the worst 5 outcomes together, giving a genuine sense of "how catastrophic does the tail typically get," not just where it starts.

PLAIN ENGLISH — WHY CVAR IS PREFERRED

Imagine two insurance policies with an identical 95% VaR of \$80 (a 5% chance of a loss exceeding \$80). Policy A's worst-case losses, when they happen, are usually around \$85. Policy B's worst-case losses, when they happen, are usually around \$5,000 (one truly catastrophic tail). VaR alone cannot tell these two policies apart at all — both report exactly "\$80." CVaR immediately reveals the difference, because it looks PAST the threshold and averages what actually happens back there. CVaR also has a mathematical property called **coherence** (technically, "subadditivity"): combining two risky positions into one portfolio should never make your risk measure go UP relative to holding them separately (this is exactly what diversification is supposed to achieve) — CVaR always respects this; VaR, in certain unusual situations, can actually violate it.

THE PROJECT'S HEADLINE NUMBERS

Asset	Hedge model	CVaR(95%) of hedging loss
SPY	Black-Scholes (blind to jumps)	\$38.86
SPY	Merton (jump-aware)	\$31.89
BTC-USD	Black-Scholes (blind to jumps)	\$10,025.16
BTC-USD	Merton (jump-aware)	\$8,184.50

In plain terms: a market maker hedging a short call with Black-Scholes Delta, in a market that actually has the jump behaviour these two real assets have shown historically, should EXPECT their worst 5%-of-the-time outcomes to average about 22% worse than a market maker using a jump-aware Merton hedge instead — a meaningful amount of tail risk the Black-Scholes-only market maker's own risk model cannot see coming.

WHERE THIS LIVES IN THE PROJECT

`src/risk_analysis.py`, functions `var_historical()` and `cvar_historical()`, and `compare_hedge_models_under_true_dgp()`, which runs the entire "simulate thousands of paths, hedge through every one, compute CVaR" pipeline for a chosen list of candidate hedge models at once.

How Sure Are We? Bootstrap Statistics and Hypothesis Testing

"CVaR(95%) = 38.86" is a single number from one particular batch of 8,000 simulated paths. How much would it change if we re-ran the simulation? And is the gap between two hedge models real, or noise?

7.1 Every Estimate Has Uncertainty

If this project's simulation were re-run with a different random-number seed, generating a DIFFERENT 8,000 simulated paths, the resulting CVaR would come out to a slightly different number — not exactly 38.86, but something close to it. A responsible risk estimate should always be accompanied by a sense of "how much could this number plausibly have been, just from the luck of which particular random paths got simulated?" This is exactly what a **standard error** and **confidence interval** quantify.

7.2 The Bootstrap: "What If We Resample Our Own Data?"

The most direct way to see how much an estimate could plausibly vary would be to re-run the ENTIRE simulation many times over and look at how much the answer moves around — but that is extremely expensive. The **bootstrap** (Efron & Tibshirani, 1993) is a remarkably simple trick that gets nearly the same information from the data you ALREADY have, with no new simulation required.

PLAIN ENGLISH

Suppose you have 8,000 simulated hedging outcomes. To bootstrap, you build a brand-new, ALSO-8,000-outcome sample by drawing 8,000 times, WITH REPLACEMENT, from your original 8,000 numbers — meaning some original outcomes might get picked two, three, or zero times in this new sample, purely by chance. You then recompute CVaR on this resampled batch. Do this a few thousand times, each time drawing a fresh random resample and recomputing CVaR, and you build up a whole DISTRIBUTION of "plausible CVaR values, given the natural randomness in which 8,000 outcomes you happened to observe." The SPREAD of that distribution is precisely an honest estimate of how uncertain your original single CVaR number really is — without ever needing to run the (expensive) full hedging simulation again.

FORMULA — BOOTSTRAP CONFIDENCE INTERVAL

$CI_{95\%} = [2.5\text{th percentile}, 97.5\text{th percentile}]$ of the bootstrap-resampled CVaR values

B The number of bootstrap resamples drawn (2,000–3,000 in this project) — more resamples give a smoother, more precise picture of the uncertainty, at the cost of more computation.

Taking the middle 95% of the resulting B bootstrap CVaR values (i.e., trimming off the most extreme 2.5% on each end) gives a 95% confidence interval: a range we can be reasonably confident contains the TRUE CVaR, if we had somehow been able to simulate infinitely many paths instead of just 8,000.

WORKED EXAMPLE — VALIDATING THE BOOTSTRAP ITSELF

Before trusting this technique on the project's real results, it was checked against a case where the TRUE answer is exactly, mathematically known: losses drawn from a plain bell curve (Normal distribution) have a CVaR with a known closed-form formula. Bootstrapping a simulated sample from that bell curve produces a confidence interval that (a) actually contains the true, known CVaR value, (b) gets narrower as the sample size grows (more data, less uncertainty, exactly as expected), and (c), across 150 independently repeated trials, contains the true value roughly 94% of the time against a 95% target — confirming the method is not badly miscalibrated.

7.3 The Paired Bootstrap Test: Is the Gap Between Two Hedges Real?

The project's central claim is a COMPARISON: "the BS hedge's CVaR is higher than the Merton hedge's CVaR." Since both hedge models are tested on the exact SAME 8,000 simulated paths (only the hedging strategy differs, not the underlying scenarios), a smarter version of the bootstrap can be used: instead of resampling each model's outcomes independently, resample the SAME set of path numbers each time, and apply that identical resample to BOTH models at once.

PLAIN ENGLISH

If path #4,213 happened to be a big historical-style crash scenario, BOTH the BS hedge and the Merton hedge experienced that exact same crash on that path — they are naturally linked ("paired") together, path by path. Resampling paired path INDICES (rather than resampling each model's results separately) correctly keeps that link intact in every bootstrap redraw, which turns out to give a much more powerful, more sensitive test of whether the GAP between the two models is real, since it isolates exactly the paths where the two hedges genuinely disagreed.

7.4 p-Values and "Statistical Significance," in Plain English

A **p-value** answers one specific question: *"if there were truly NO real difference between the BS hedge and the Merton hedge, how likely would we be to see a gap in our sample at least as large as the one we actually observed, purely by chance?"* A small p-value (conventionally, below 0.05, i.e. below 5%) means "very unlikely to be pure chance" — which is what people mean when they say a result is **statistically significant**.

THE PROJECT'S RESULT

Asset	BS CVaR [95% CI]	Merton CVaR [95% CI]	Paired difference [95% CI]	p-value
SPY	38.86 [37.10, 40.56]	31.89 [30.27, 33.48]	6.97 [6.75, 7.17]	< 0.001
BTC-USD	10,025.16 [9,575.06, 10,487.01]	8,184.50 [7,748.34, 8,622.20]	1,840.66 [1,790.01, 1,895.07]	< 0.001

Both gaps are significant well beyond the conventional 5% threshold — in fact so far beyond it that, across thousands of bootstrap resamples, essentially none of them showed the gap reversing sign. Note also that the confidence intervals on BS-hedge CVaR and Merton-hedge CVaR (columns 2 and 3) DO NOT EVEN OVERLAP — a visually obvious, not just formally significant, difference.

WHERE THIS LIVES IN THE PROJECT

`src/statistical_inference.py`, functions `bootstrap_ci()` (single-sample confidence interval) and `paired_bootstrap_test()` (the paired comparison described above).

Extreme Value Theory — Estimating Risks You Haven't Seen Yet

How can you responsibly estimate "how bad could the worst 1-in-1,000 outcome be" when you've only simulated 8,000 paths total?

8.1 The Deep-Tail Data Problem

Historical CVaR (Part 6) works by literally sorting the simulated outcomes and averaging the worst few. At the 95% level ($q=0.05$) with 8,000 paths, that means averaging the worst 400 outcomes — plenty of data to get a stable answer. But at the 99.9% level ($q=0.001$), that same 8,000-path sample only has about 8 observations informing the estimate. Eight data points is a very thin basis for a number a risk manager might want to rely on.

PLAIN ENGLISH

Imagine trying to estimate how bad the worst flood in 1,000 years could be, but you only have 8 years of rainfall records. You obviously cannot just look at your single worst recorded year and call it "the 1-in-1,000-year flood" — you need some kind of principled way to extrapolate BEYOND what you've actually observed. Extreme Value Theory (EVT) is exactly that principled extrapolation method, applied to financial losses instead of floods.

8.2 An Almost Magical Theorem About Tails

You may already know the **Central Limit Theorem**: no matter what weird distribution a single random variable follows, the AVERAGE of many independent copies of it tends toward the same familiar bell curve (Normal distribution) as you average more and more of them. Extreme Value Theory rests on an equally remarkable, less famous cousin of that theorem, due to Pickands (1975) and Balkema & de Haan (1974): no

matter what distribution your data follows, the shape of the EXCEEDANCES over a sufficiently high threshold (how much worse than the threshold the bad outcomes get, when they do exceed it) always converges to one specific family of distributions: the **Generalized Pareto Distribution (GPD)**.

FORMULA — THE GENERALIZED PARETO DISTRIBUTION

$$P(\text{Loss} > \text{threshold} + y \mid \text{Loss} > \text{threshold}) = (1 + \xi y / \beta)^{-1/\xi}$$

threshold	A high cutoff level chosen in advance (this project defaults to the 90th percentile of simulated losses — the worst 10%).
y	An "exceedance" amount — how much further past the threshold a particular bad outcome went.
ξ	("xi") The GPD's shape parameter — controls how "fat" the extreme tail is. Larger ξ means a heavier, more dangerous tail (more chance of a truly enormous loss); ξ=0 is the special case of an exponentially-thinning tail.
β	("beta") The GPD's scale parameter — sets the typical SIZE of exceedances, in dollar terms.

Both ξ and β are fitted to the observed exceedances over the threshold using maximum likelihood (a standard statistical fitting procedure: find the parameter values that make the ACTUALLY OBSERVED exceedances as probable as possible under the GPD formula). Once fitted, the GPD formula can be mathematically inverted to estimate VaR/CVaR at quantiles FAR beyond what the raw data can inform directly — this is the entire point.

WORKED EXAMPLE — VALIDATING EVT ON DATA WITH A KNOWN ANSWER

Before trusting this on the project's real results, EVT was tested on synthetic data specifically built with a known, exact tail: ordinary "calm day" losses below a threshold, plus a separate, deliberately-inserted tail of exceedances drawn EXACTLY from a GPD with pre-chosen, known ξ and β. Repeated over 40 independent re-draws, at a very deep quantile (0.05%, meaning only about 3 observations out of several thousand would even be in that tail historically): the EVT-based estimate was closer to the TRUE, known answer on AVERAGE (6.0% mean relative error vs. 7.4% for the plain historical estimator) and, for CVaR specifically, notably MORE STABLE from one re-draw to the next (lower variance) — precisely the regime EVT is designed to help with.

THE PROJECT'S ACTUAL RESULTS

Asset / hedge	ξ	β	Historical CVaR(99.9%)	EVT CVaR(99.9%)
SPY / BS hedge	0.012	12.22	86.52	89.03
SPY / Merton hedge	0.047	10.70	77.01	81.85
BTC-USD / BS hedge	0.053	2,989.03	25,836.58	24,287.93
BTC-USD / Merton hedge	0.070	2,703.31	23,411.42	21,936.47

At the moderate 95% level, historical and EVT estimates agree closely (both have ample data); at this much deeper 99.9% level, they diverge — exactly where the historical estimator is running low on data and EVT is instead extrapolating smoothly from the fitted tail shape.

8.3 Hill's Estimator: A Simpler Cross-Check, With a Known Weakness

Hill's (1975) estimator is a simpler, older way to estimate the tail shape parameter ξ , using only the ratio of the largest few observed losses to each other. It is validated in this project on its ideal textbook case (data whose tail is EXACTLY a pure power law, called a Pareto distribution), where it works excellently.

CAUTION / LIMITATION — REPORTED HONESTLY

When tested on a Generalized Pareto tail (rather than a pure Pareto tail), Hill's estimator was found to carry a genuine, known finite-sample bias — it is a RATIO-based formula, and ratios are sensitive to an added constant (the threshold) in a way that a pure power-law tail (with no such added constant) isn't. Rather than hide this or quietly drop the method, this project keeps Hill's estimator as a secondary STABILITY/plausibility diagnostic (via a "Hill plot": the estimate recomputed across a range of possible sample sizes, which should look roughly flat if reliable) alongside the primary, more carefully validated GPD maximum-likelihood fit — not as a replacement for it.

WHERE THIS LIVES IN THE PROJECT

`src/tail_risk.py`, functions `fit_gpd_pot()` (the main GPD fit), `evt_var()` / `evt_cvar()` (inverting the fitted tail to a risk estimate), `hill_estimator()`, and `mean_excess_plot_data()` (a second graphical diagnostic for choosing a sensible threshold, shown in the dashboard's Tail Risk tab).

American Options — Binomial Trees and the Crank-Nicolson PDE

Every model in Part 3 prices European options only. Real listed stock options are usually American-style. Here is how to price those, using two completely independent numerical methods.

9.1 European vs. American: The Right to Exercise Early

A **European** option (the only kind priced in Parts 3–8) can only be exercised on its exact maturity date — you either use it at expiry or let it expire worthless. An **American** option grants the extra right to exercise it **EARLY**, at any point before maturity, if that happens to be advantageous. (Confusingly, these names have nothing to do with geography — they are just historical labels.)

PLAIN ENGLISH — WHEN WOULD EARLY EXERCISE EVER MAKE SENSE?

For a **put** option on a non-dividend stock, it can become worthwhile to exercise early if the stock has fallen so far that the option is deep "in the money": exercising now locks in the strike proceeds and lets you start earning interest on that cash immediately, rather than waiting for maturity to collect the same (or a similar) payoff. For a **call** option, early exercise is essentially **NEVER** worthwhile on a stock that pays no dividends (you'd be giving up the remaining time value for nothing) — but it **CAN** become worthwhile just before a big dividend payment, because dividends are paid to whoever owns the actual stock, not to option holders, and the stock price typically drops by roughly the dividend amount on the payment date. Exercising the call just before that date lets you capture the dividend instead of watching the stock (and your un-exercised option) drop in value for a payment you never received.

WHY THIS MATTERS FOR THIS PROJECT

Every one of this project's three pricing models (Black-Scholes, Heston, Merton) only prices EUROPEAN options — a genuine capability gap, since real listed single-stock equity options are American-style, and "price an American option" is one of the single most common topics tested in quantitative finance interviews.

`american_options.py` closes this gap.

9.2 The Binomial Tree: A Simple, Visual Approach

The Cox-Ross-Rubinstein (1979) binomial tree replaces the smooth, continuous price motion of Geometric Brownian Motion with something much simpler to picture: at each small time step, the price can only do one of two things — move UP by a fixed multiplicative factor u , or move DOWN by a fixed factor d . Repeating this many times over builds a branching "tree" of every possible price path.

FORMULA — SETTING UP THE TREE

$$u = e^{\sigma\sqrt{dt}} \quad d = 1/u \quad p = (e^{r \cdot dt} - d) / (u - d)$$

dt	The size of one small time step (e.g., one trading day out of the option's life).
u, d	The up and down multiplicative price factors — chosen (via this exact formula) so the tree's spread of possible prices matches the model's volatility σ as the number of steps grows.
p	The "risk-neutral probability" of an up-move — NOT the real-world probability of the price rising, but a mathematical device chosen so that the tree is internally consistent with a no-arbitrage (no free lunch) assumption.

Step-by-step: how the tree is actually used to find a price

1. **Build forward.** Starting from today's price, repeatedly apply u or d to enumerate every possible price at every future time step, all the way out to maturity — this creates a branching (but "recombining," since an up-then-down move lands on the same price as a down-then-up move) lattice of prices.
2. **Compute the payoff at every final (maturity) node** — simply $\max(S-K, 0)$ for a call, using whatever price that node represents.
3. **Work BACKWARD, one time step at a time.** At every node one step before maturity, the option's value is the DISCOUNTED, probability-weighted average of its two possible next-step values: $p \cdot (\text{value if up}) + (1-p) \cdot (\text{value if down})$, all multiplied by $e^{-r \cdot dt}$ to discount back one step.
4. **For an American option, at every node, also check immediate exercise.** Compare the "continuation value" just computed in step 3 against the payoff of exercising RIGHT NOW at that node's price, and keep whichever is LARGER — a rational holder always takes the better of the two options.
5. **Repeat step 3–4 all the way back to today** — the value at the very first (starting) node is the option's price.

PLAIN ENGLISH

Picture a branching tree of possible futures, drawn out completely, with the option's known, certain value written at every branch tip (maturity). Working backward one layer at a time, at every branching point you ask "given the two possible next outcomes, and their probabilities, what is this branch worth right now?" — and for an American option, you ALSO ask "would I be better off just cashing out immediately, right here, instead of waiting?" and always take the better of those two numbers. By the time you've worked all the way back to the trunk of the tree, you have today's fair price, AND you've automatically discovered every point in the tree where early exercise would have been the right call.

9.3 The Crank-Nicolson PDE: Solving an Entire Grid at Once

A **partial differential equation (PDE)** is, in plain terms, a rule connecting how a quantity changes when you nudge ONE of its inputs to how it changes when you nudge a DIFFERENT input — here, the rule (derived by Black, Scholes, and Merton, the same insight underlying Part 3.1's formula) connects how the option's price changes as the underlying price S moves to how it changes as time passes, WITHOUT assuming ahead of time what the price actually looks like.

PLAIN ENGLISH

Instead of building one branching tree of individual price scenarios (like Part 9.2), the PDE approach lays out a two-dimensional GRID: one axis is the underlying's price (chopped into, say, 200 discrete price levels from \$0 up to several times today's price), and the other axis is time (chopped into, say, 200 discrete days from today out to maturity). We know the option's exact value along the "maturity" edge of this grid (it's just the payoff). The PDE rule then lets us fill in the value at every OTHER point of the grid, one column of time-steps at a time, working backward from the known maturity edge — using simple algebra at each grid point based on its neighbours, rather than simulating individual random paths at all.

Why "Crank-Nicolson," and why solve a whole grid instead of one price?

"Crank-Nicolson" (1947) refers to a specific, numerically well-behaved recipe for turning the PDE's smooth calculus into simple grid arithmetic (it averages two different, more naive ways of doing this — called "explicit" and "implicit" schemes — to get a method that is both accurate and numerically stable). The genuinely useful side-benefit of solving an entire price/time GRID, rather than one option price at a time, is that a single solve gives you:

- **The option's price at every possible starting spot price**, not just today's actual spot price — useful for building an entire smile/surface of prices at once.
- **Delta and Gamma, essentially for free**, just by looking at the grid values immediately NEXT TO today's actual price and using simple slope arithmetic — no separate, extra re-pricing needed at all.
- **The early-exercise boundary**: at every time step, there is some critical price below which (for a put) immediate exercise is optimal. Tracking exactly where that boundary sits at every single time step traces out a curve over the whole life of the option — visualized directly in the dashboard (Part 13).

FORMULA — ENFORCING EARLY EXERCISE ON THE GRID

$$V_{\text{grid}} \leftarrow \max(V_{\text{grid}}, \text{payoff}_{\text{grid}}) \quad (\text{applied after every single time step})$$

This is the entire mechanism for American-ness in the PDE approach: after computing what the option would be worth if you were NOT allowed to exercise early (the ordinary PDE step), simply overwrite any grid point where immediate exercise would have been worth MORE — a direct, grid-wide analogue of the binomial tree's node-by-node "compare and keep the larger" rule from Part 9.2.

THE PROJECT'S VALIDATION AND RESULTS

Asset	European put	American put (tree)	Early-exercise premium	Crank-Nicolson price
SPY ($\sigma=15.8\%$)	\$18.14	\$18.73	\$0.58 (3.22%)	\$18.53
BTC-USD ($\sigma=45.6\%$)	\$7,545.86	\$7,612.82	\$66.96 (0.89%)	\$7,606.73

The two completely independent numerical methods (tree and PDE) agree to within a few cents across a range of strikes. Both are also confirmed to satisfy every piece of theory described in Part 9.1: an American call with NO dividends exactly matches its European price (early exercise adds nothing); an American call WITH a dividend shows a genuine positive early-exercise premium; and the American put's early-exercise boundary always stays below the strike price, as it logically must.

WHERE THIS LIVES IN THE PROJECT

`src/american_options.py` : `binomial_tree_option()` (the tree, vectorized across all nodes at a given time step for speed) and `crank_nicolson_american()` (the PDE, solved using a fast "tridiagonal" linear-algebra solver at each time step, and optionally tracking the early-exercise boundary as it goes).

Monte Carlo Simulation and Variance Reduction

The technique used everywhere in this project to cross-validate pricing formulas and to run the headline hedging experiment itself — explained from the ground up, plus three ways to make it more efficient.

10.1 What Is Monte Carlo Simulation?

Monte Carlo simulation (named after the famous casino, because of its reliance on repeated random chance) is a way of estimating a mathematical answer — here, an option's fair price, or the CVaR of a hedging strategy — by generating a very large number of random "what if" scenarios on a computer and simply averaging the results, rather than solving an equation exactly.

PLAIN ENGLISH

Suppose you wanted to know the average outcome of rolling two dice and adding them together, but had somehow forgotten how to compute it directly. You could instead roll two real dice ten thousand times, add up the two numbers each time, and average all ten thousand sums — you would get a number extremely close to the true mathematical average (7), without ever writing down a formula. Monte Carlo option pricing does exactly this, but "rolling the dice" means generating a random future price PATH using one of Part 3's models (a sequence of random up/down/jump shocks, one per day, compounded together), and the "outcome" is the option's payoff at the end of that particular simulated path.

FORMULA — MONTE CARLO PRICING

$$\text{Price} \approx e^{-rT} \cdot (1/N) \cdot \sum_{i=1}^N \text{payoff}(S_T^{(i)})$$

N The number of independently simulated paths (this project typically uses tens of thousands for pricing cross-checks, and 8,000 for the main hedging study).

$S_T^{(i)}$ The final, simulated price at maturity for the i-th random path.

Simply average the discounted payoff across all N simulated paths — the **Law of Large Numbers** (a foundational theorem of probability) guarantees this average converges to the TRUE price as N grows, and the **standard error** of that average (how far off a finite-N estimate is likely to be) shrinks proportionally to $1/\sqrt{N}$ — quadrupling N only halves your uncertainty, which is why getting extra precision "for free" (variance reduction, below) is genuinely valuable rather than a minor optimization.

10.2 Antithetic Variates: Pairing Each Path With Its Mirror Image

The simplest variance-reduction trick: for every random path simulated using a particular sequence of random shocks, ALSO simulate a second "mirror image" path using the exact NEGATIVE of those same random shocks, and average the two payoffs together as a single combined observation.

PLAIN ENGLISH

A random shock and its exact opposite are, by construction, equally likely draws from the same bell curve. If your first simulated path happens to wander unusually far upward (just by chance), its mirror-image partner will wander unusually far downward by the identical amount — averaging the two largely cancels out that particular unlucky/lucky draw's contribution to your final estimate's noise, essentially for free, since generating the mirror image costs almost nothing extra once you already have the original random numbers.

THE PROJECT'S RESULT

Tested at an EQUAL total simulation budget (same total number of price paths either way), antithetic variates reduced the Monte Carlo standard error of the Merton price estimate by 10.5% for SPY and 12.6% for BTC-USD — a real, "more precision for the same computer time" improvement, with the estimate remaining just as unbiased (still agreeing with the independently-validated closed-form Merton price) as plain, unpaired simulation.

10.3 Control Variates: Borrowing a Known Answer

The second technique exploits the fact that this project already has an EXACT, closed-form Black-Scholes price (Part 3.1) available. Simulate a Merton path and a "pretend there are no jumps" Black-Scholes path **using the exact same random diffusive shocks** for both, so the two simulated payoffs are highly correlated with each

other. Then correct the Merton Monte Carlo estimate by however much the SAME-random-numbers Black-Scholes simulation happened to miss its own KNOWN, exact answer.

PLAIN ENGLISH

Imagine two friends, A and B, running the same errand route on the same windy day. You already know EXACTLY how long B's route should take under calm conditions (a known answer). If B actually arrives 4 minutes late today (because of the wind), and A took a very similar, correlated route, you can reasonably guess A was ALSO slowed by about 4 minutes of wind — and subtract that known wind effect from A's observed time to get a better estimate of A's "no-wind" time than A's raw observed time alone would give you. Control variates do exactly this: the Black-Scholes simulation's known error (from the "wind" of that day's particular random numbers) is used to correct the Merton simulation's estimate, since both experienced the identical randomness.

THE PROJECT'S RESULT

Control variates reduced Monte Carlo standard error by 36.8% for SPY and 42.3% for BTC-USD at equal simulation budget — a substantially bigger improvement than antithetic variates, because the Black-Scholes/Merton correlation this method exploits is very strong (the two models differ only by the presence of jumps, which are relatively rare events).

10.4 Importance Sampling: Deliberately Oversampling the Bad Outcomes

The previous two techniques speed up ordinary option PRICING. This one is purpose-built for a different, arguably more relevant problem in this project: efficiently estimating a very deep-tail CVaR (say, the 99% level) without needing an enormous number of paths.

PLAIN ENGLISH

Imagine testing an extremely reliable lightbulb for its failure rate. Testing 10,000 completely typical bulbs at random will likely produce almost no failures at all — you've "wasted" most of your test budget confirming that ordinary bulbs don't fail, learning very little about the RARE failure cases you actually care about. A smarter approach: deliberately test bulbs under harsher-than-normal conditions (more oversampling of "bad" scenarios), see how they fail under those harsher conditions, and then mathematically correct ("reweight") your results to reflect what the TRUE failure rate would be under normal conditions. This project applies the identical idea to crash risk: instead of simulating paths under the asset's TRUE, calibrated jump frequency (where crashes are rare and a large sample is needed to see enough of them), it deliberately INFLATES the jump arrival rate — simulating a deliberately crash-heavy alternate world — then mathematically reweights every simulated path to correct for the fact that crashes were made artificially more common.

FORMULA — THE REWEIGHTING ("LIKELIHOOD RATIO")

$$L = e^{\lambda T(c-1)} \cdot c^{-N}$$

c	The "tilting" factor by which the jump rate is deliberately inflated for simulation purposes (this project uses $c=2$, i.e., simulating with DOUBLE the true jump frequency).
N	The actual number of jumps that occurred on one particular simulated path (under the inflated rate).
L	The "correction weight" for that path: paths with MORE jumps than typical (which are now artificially overrepresented, since we inflated the jump rate) get a SMALLER weight when we average results back together, exactly undoing the artificial oversampling.

Every simulated outcome is multiplied by its own L before being averaged into a final CVaR estimate — a technique called **self-normalized importance sampling**, which produces a provably unbiased final answer despite simulating from a deliberately "wrong" (tilted) process along the way.

THE PROJECT'S RESULT

Asset	Ground truth (n=8,000, true rate)	Plain MC (n=2,000)	Importance-sampled (n=2,000)
SPY CVaR(99%)	52.07	52.97	49.15
BTC-USD CVaR(99%)	13,447.05	13,818.76	13,672.87

Validated two ways: the importance-sampled estimate using only 2,000 paths lands close to a ground-truth answer that needed a 4×-larger, 8,000-path plain simulation to establish; and across 20 independently repeated re-draws at the SAME (small) path count, importance sampling showed lower estimator variance than plain Monte Carlo specifically at this deep quantile — exactly the claimed benefit.

CAUTION / LIMITATION — VERIFIED, NOT ASSUMED

The tilting factor c is a genuine tradeoff, not a dial to be cranked up indefinitely: too LITTLE tilting barely oversamples the tail at all (little benefit over plain simulation), while too MUCH tilting creates a small number of paths with enormous correction weights L , which can actually INCREASE the estimator's variance again (a phenomenon called "weight degeneracy"). This project verified empirically that $c=2$ works well for its specific problem, while $c=4$ or higher was measurably WORSE than plain Monte Carlo on the exact same problem — the right amount of tilting must be checked, not assumed.

WHERE THIS LIVES IN THE PROJECT

```
src/variance_reduction.py : merton_price_antithetic() , merton_price_control_variate() ,  
merton_simulate_paths_importance_sampled() , and weighted_var_cvar() (a version of CVaR/VaR that correctly  
accounts for each simulated path carrying its own reweighting factor, rather than every path counting equally).
```

P&L Attribution, Stress Testing, and Model Validation

Three more tools a real bank's market-risk desk uses every single day, each corroborating the headline finding from a completely different angle.

11.1 Greeks-Based P&L Attribution: "Where Did Today's Money Go?"

Real trading desks don't just report "we made/lost \$X today" — they EXPLAIN it, breaking the day's change in the option's value into pieces attributable to each Greek from Part 3.2, plus whatever is left over.

FORMULA — THE P&L EXPLAIN

$$\Delta V \approx \Delta \cdot \Delta S + \frac{1}{2} \Gamma \cdot \Delta S^2 + \Theta \cdot \Delta t + \text{residual}$$

ΔV	The ACTUAL total change in the option's value from one day to the next.
$\Delta \cdot \Delta S$	The piece explained by Delta: how much the option's value should change, JUST from the underlying moving by ΔS , holding Delta itself fixed.
$\frac{1}{2} \Gamma \cdot \Delta S^2$	A correction piece explained by Gamma: since Delta itself isn't really fixed as the price moves, this term accounts for how much MORE (or less) the price change was worth because Delta was curving as it went.
$\Theta \cdot \Delta t$	The piece explained by ordinary time decay — the option loses (or occasionally gains) a predictable amount of value just from one day passing, even if the price hadn't moved at all.
residual	Whatever is LEFT OVER after subtracting the three predictable pieces above from the actual change — this is exactly what a smooth, gradual (Delta/Gamma/Theta-based) approximation could NOT anticipate, including any actual price JUMP that occurred that day.

PLAIN ENGLISH

This formula is a "Taylor expansion" — a standard mathematical trick for approximating how a smooth, curved function changes using only its slope (Delta), its curvature (Gamma), and its dependence on time (Theta), evaluated at a single starting point. It works very well for ORDINARY, gradual price moves. But on a day when the underlying price genuinely JUMPS, no smooth approximation built from a single starting point can see that coming — so the "residual" term spikes sharply on exactly those days, which is precisely the point: it turns this project's headline CVaR finding into a day-by-day, causal story instead of only a single terminal number.

THE PROJECT'S FINDING

Aggregated across many simulated paths under the SAME calibrated, genuinely-jumpy "true" market, the Black-Scholes hedge's residual distribution shows a visibly FATTER left tail (a higher residual CVaR) than the Merton hedge's — the same headline finding as Part 6, now demonstrated at the level of individual trading days, and directly attributable to jump days specifically (a synthetic test in the module inserts one large artificial jump mid-path and confirms the residual spikes almost exactly on that day, and nowhere else).

WHERE THIS LIVES IN THE PROJECT

`src/pnl_attribution.py`, functions `compute_pnl_attribution()` (one path, full day-by-day detail) and `compute_pnl_attribution_batch()` (many paths at once, summary totals only).

11.2 Deterministic Scenario Stress Testing

Every method so far is **statistical**: it works from a probability distribution built out of thousands of randomly simulated paths. Real risk functions ALSO run **deterministic stress tests** — specific, hand-designed "what if THIS exact thing happened" scenarios, with no reference to any probability distribution at all. This matters because "this precise scenario never showed up in my calibration window" is exactly the kind of risk a purely statistical model can under-price.

Scenario	What it represents
Black Monday	A single-day, 20% crash (echoing October 19, 1987).
COVID-style crash	A multi-day crash followed by an elevated-volatility aftermath (echoing March 2020).
Flash crash	A sharp intraday-style drop with a partial same-window rebound.
Vol-regime doubling	No directional crash at all — just realized volatility suddenly doubling.
Slow grind lower	Sustained mild NEGATIVE drift with LOW volatility — the "boring but bleeding" scenario a jump-focused model can miss entirely.
Melt-up rally	A sustained sharp RALLY — testing that a short-option position's pain isn't only ever about crashes.

THE PROJECT'S FINDING

Every one of these scenarios is run through the EXACT SAME hedging engine as the statistical study (Part 5) — not a separate, ad-hoc calculation — and the Merton (jump-aware) hedge outperforms the Black-Scholes hedge in EVERY single named scenario, on both assets, with by far the largest gaps in the genuine crash scenarios (on BTC-USD, the Merton hedge's Black-Monday loss is more than 5× smaller than the BS hedge's). This is a completely distribution-free corroboration of the statistical finding from Part 6.

WHERE THIS LIVES IN THE PROJECT

```
src/stress_testing.py , functions build_scenarios() (constructs each named price path) and  
run_stress_scenarios() (runs every scenario through the same hedging engine used everywhere else).
```

11.3 Model Validation: Does the VaR Model Actually Work Out-of-Sample?

Before any risk model is trusted at a real bank, an independent "model-validation" function checks whether it is even well-calibrated: if a VaR model claims "only a 5% chance of losing more than \$X," does that actually hold up against real, subsequent outcomes the model was NOT trained on?

PLAIN ENGLISH — THE HONEST TRAIN/TEST SPLIT

It would be cheating to estimate your VaR threshold using ALL your data, then check how well that same threshold "predicts" the very data it was built from — of course it will look reasonable, because it was fit to that exact data. This project instead splits its simulated paths in half: the FIRST half is used only to estimate the VaR threshold, and the SECOND, held-out half is used only to COUNT how many times that threshold was actually breached — an honest, out-of-sample test, mirroring exactly how a bank's model-validation team is required to test a production risk model.

FORMULA — KUPIEC'S PROPORTION-OF-FAILURES TEST

A likelihood-ratio statistical test comparing the ACTUALLY OBSERVED breach rate in the held-out half against the model's STATED target rate (e.g., 5%). Its output is, once again, a p-value (Part 7.4): a small p-value means the observed breach rate is significantly different from what the model promised — i.e., the model is miscalibrated, not just unlucky.

Basel traffic light A simpler, regulator-friendly version of the same idea: count the number of breaches in the held-out sample, and classify the model as GREEN (a healthy number of breaches), YELLOW (a borderline/concerning number), or RED (clearly too many breaches to trust the model) based on standard, published cutoffs.

THE PROJECT'S FINDING

Asset	Hedge model	Exception rate (target 5%)	Kupiec p-value	Basel zone
SPY	BS hedge	5.65%	0.064	Yellow
SPY	Merton hedge	5.55%	0.117	Green
BTC-USD	BS hedge	5.12%	0.718	Green
BTC-USD	Merton hedge	4.95%	0.884	Green

A genuinely interesting, honestly-reported finding: the BS hedge's OWN self-assessed VaR model for SPY lands in the Basel YELLOW zone (borderline miscalibrated out-of-sample), while the Merton-hedge model is solidly GREEN. This is exactly the kind of red flag that would trigger a model refinement request from an independent model-validation function at a real bank.

WHERE THIS LIVES IN THE PROJECT

`src/model_validation.py` , functions `kupiec_pof_test()` , `basel_traffic_light()` , and `run_var_backtest()` (which performs the train/held-out split and runs both tests).

The Experiment Pipeline — A 14-Step Walkthrough

Running `src/run_experiment.py` executes every idea in Parts 3–11 in one continuous script. Here is what happens, step by step, and why the steps are ordered this way.

All of this project's tables and figures (in `results/tables/` and `results/figures/`) are produced by one script, `src/run_experiment.py`, which runs 14 steps in sequence, each one building on data prepared by an earlier step. The whole pipeline completes in well under two minutes on cached historical data.

Step	What it does	Concepts from
1	Load real SPY and BTC-USD price history and calibrate both Heston and Merton parameters to each asset.	Part 4
2	Plot each model's implied-volatility "smile" across strikes, as a pricing sanity check — even with one flat input volatility, Heston and Merton naturally produce a smile shape that Black-Scholes cannot.	Part 3
3	The headline experiment: simulate 8,000 alternate 3-month futures under each asset's calibrated Merton process, hedge daily with either BS or Merton Delta, and compute CVaR of the resulting hedging losses.	Parts 5, 6
4	Repeat Step 3's comparison, now also including a Heston hedge, at the SAME full 8,000-path scale (made possible by the COS method).	Parts 3.3–3.4
5	Sweep the rebalancing frequency from daily to monthly, tracking the cost/risk tradeoff.	Part 5.2
6	The real-data backtest: replay actual historical price windows (not simulated paths), using WALK-FORWARD calibration to avoid look-ahead bias.	Parts 4, 6
7	Compare fixed-frequency rebalancing against band ("no-transaction region") hedging.	Part 5.3
8	Run all six named deterministic stress scenarios through the same hedging engine.	Part 11.2
9	Backtest the VaR model itself using the honest train/held-out split, Kupiec's test, and the Basel traffic light.	Part 11.3
10	Compute bootstrap confidence intervals and a paired hypothesis test on the Step 3 headline numbers.	Part 7
11	Fit the Extreme Value Theory tail model and compare it against historical CVaR at several tail depths.	Part 8
12	Run the Delta-Vega hedging comparison under each asset's own calibrated, genuinely stochastic-volatility Heston process.	Part 5.4
13	Price American options (binomial tree + Crank-Nicolson PDE) and produce the early-exercise boundary figure.	Part 9
14	Demonstrate Monte Carlo variance reduction (antithetic/control variates) and importance sampling for efficient tail-risk estimation.	Part 10

PLAIN ENGLISH — WHY THIS ORDER?

The pipeline is ordered so that later steps can always reuse results computed by earlier ones (Step 1's calibration feeds every single later step; Step 3's headline paths are re-used, with the identical random seed, by Steps 10 and 11 so that the bootstrap confidence intervals and EVT tail model describe EXACTLY the same numbers quoted as the headline result, not a separately-drawn approximation to them). Steps 1–9 form the "original" core experiment; Steps 10–14 are the newer additions layered on top, each one deepening or stress-testing a specific piece of the original story rather than changing its central conclusion.

WHERE THIS LIVES IN THE PROJECT

`src/run_experiment.py` , one function per step (named `step1_load_and_calibrate()` through `step14_variance_reduction()`), called in order from a single `main()` function at the bottom of the file. Every table this pipeline produces is saved as a CSV file in `results/tables/` , and every figure as a PDF in `results/figures/` — both directories are also where the interactive dashboard (Part 13) loads its precomputed, "report-scale" numbers from, so what you see in the dashboard and what is quoted in `report/report.md` are always the same underlying numbers.

Inside the Dashboard — All 16 Tabs Explained

Run [this link](#) to open this interactive companion in your browser. This part explains every single tab: what it shows, what you can control, and how to read it.

The dashboard is built with **Streamlit**, a Python library that turns a normal Python script into an interactive web page: every slider, dropdown, and button you see is one line of Python code, and every time you change a control, Streamlit silently re-runs the relevant calculation and refreshes the charts — most of which are built with **Plotly**, a charting library whose charts you can zoom into, pan around, hover over for exact values, and toggle individual data series on/off by clicking their legend entry. This is deliberately NOT a "configure everything, then click one Run button" tool: every tab is built to show results across MULTIPLE parameter combinations simultaneously (different hedge models, different rebalancing frequencies, both assets at once), so you can explore the model-risk story yourself rather than trusting one precomputed chart.

PLAIN ENGLISH — "PRECOMPUTED" VS. "LIVE" NUMBERS

You will see two flavours of result throughout the dashboard. **Precomputed ("report-scale") numbers** are the exact figures from `results/tables/`, generated once by the full pipeline (Part 12) at full scale (typically 8,000 simulated paths) — these are the authoritative numbers quoted in the written report. **Live numbers** are recomputed on the spot, in your browser session, using whatever parameters you've currently set on the page's sliders and dropdowns — usually at a smaller path count so the page stays responsive. Comparing the two side by side is itself a useful exercise: if the live, smaller-sample number is close to the precomputed, full-scale number, that is direct, hands-on evidence that the original result is a stable finding rather than a fluke of one particular run.

Tab 1 — Overview

The landing page: a four-box "recipe" of the whole experiment (true market → hedge model → discrete hedging simulation → CVaR of P&L), four headline metric cards comparing BS-hedge vs. Merton-hedge CVaR on both the real-data backtest and the 8,000-path simulation, and a combined bar chart showing all four numbers side by side, faceted by asset.

LOOK FOR

An expandable panel titled "Methodological check: is the real-data backtest look-ahead biased?" — this shows, side by side, the corrected walk-forward CVaR numbers (Part 4.1, Part 6) against what the ORIGINAL, look-ahead-biased full-sample calibration would have reported, quantifying exactly how much that fixed bug mattered (small for SPY, a genuinely material ~40–65% overstatement for BTC-USD).

Tab 2 — Data & Calibration

For each asset, side by side: a price history chart, a rolling 21-day realized-volatility chart, and a histogram of daily returns with the threshold-flagged "jump days" (Part 4.2) highlighted in a contrasting colour, plus a small table of every calibrated Heston and Merton parameter.

LOOK FOR

How visibly different SPY's and BTC-USD's return histograms are — BTC-USD's is noticeably wider and has more extreme flagged jump days, a direct visual confirmation of the calibrated numbers in Part 4.

Tab 3 — Live Market Check

Fetches SPY's REAL, CURRENT options chain live from Yahoo Finance (you can choose a target expiry with a slider) and overlays the market's actual, live implied-volatility smile against this project's historically-calibrated Merton model smile at the closest matching expiry — a genuine, live Q-measure vs. P-measure comparison (Part 4.1), computing the "volatility risk premium" (how much extra volatility the market is currently pricing in versus the historical model) directly.

LOOK FOR

This tab requires live internet access to Yahoo Finance and degrades gracefully with a plain warning message if that fetch fails — every other tab is unaffected either way. BTC-USD has no listed options chain on Yahoo Finance, so this tab is SPY-only.

Tab 4 — Volatility Smile

For each asset, a chart of "Black-Scholes implied volatility" across a range of strikes, for all three models. Black-Scholes' own line is perfectly flat by construction (Part 3.1 assumes one constant σ); Heston's and Merton's lines curve into a "smile" or "smirk" shape purely from their own richer dynamics, EVEN THOUGH all three models were fed the same starting flat volatility number — a visual pricing sanity check that these more complex models are actually doing something meaningfully different from Black-Scholes.

Tab 5 — Headline CVaR Result

The project's central chart: a bar chart of BS-hedge vs. Merton-hedge CVaR(95%), both assets at once (Part 6). Below it, a slider lets you choose how many paths (1,000 to 8,000) to LIVE re-simulate, rendered as violin plots (a distribution shape drawn sideways, wider where outcomes are more common) showing the full shape of the hedging-P&L distribution for each hedge model, with CVaR and mean P&L metric cards underneath each.

LOOK FOR

How the BS hedge's violin plot visibly has a longer, fatter tail stretching toward large losses compared to Merton's — the CVaR number is a summary of exactly this visual difference.

Tab 6 — Statistical Rigor

Implements Part 7 visually: a chart with one point-plus-error-bar per hedge model per asset, showing the CVaR point estimate and its 95% bootstrap confidence interval, followed by a plain-language statement of the paired hypothesis test result (significant or not, with the exact p-value) for each asset.

LOOK FOR

A "Live re-bootstrap" section lets you choose the asset, path count, number of bootstrap resamples, and tail level (q), and watch the confidence interval recompute in front of you — try lowering the path count and watch the interval visibly widen, a direct, hands-on demonstration of "less data means more uncertainty" (Part 7.1).

Tab 7 — Tail Risk (EVT)

Implements Part 8 visually: for a chosen asset, path count, and GPD threshold (adjustable with a slider), a chart comparing historical CVaR against EVT-based CVaR across a range of tail depths (5% down to 0.1%) on a logarithmic axis, plus the two classic EVT diagnostic plots: the **mean-excess plot** (should look roughly straight/linear if a GPD tail fit is appropriate) and the **Hill plot** (should look roughly flat/stable across a range of sample sizes if the tail-index estimate is trustworthy).

LOOK FOR

How historical and EVT curves overlap almost perfectly at the LEFT (shallow, 5%) end of the chart, then visibly diverge further right (deeper into the tail) — a direct visual of exactly the "historical estimator runs out of data" story from Part 8.1.

Tab 8 — P&L Attribution

Implements Part 11.1 visually. Pick an asset, hedge model, random seed, and one specific simulated path (by index), and see that single path's cumulative Delta/Gamma/Theta/residual P&L build up over time next to the actual total, plus a bar chart of each individual day's residual with the single largest day highlighted. Below that, an aggregated view across many paths shows the full DISTRIBUTION of total residual P&L for BS vs. Merton hedges as overlapping histograms.

LOOK FOR

Pick a path where the underlying happened to move a lot on one particular day, and watch that day's residual bar visibly spike — the day-by-day, "smoking gun" evidence for jump risk described in Part 11.1.

Tab 9 — Heston Finding

A bar chart comparing BS, Heston, and Merton hedge CVaR, now confirmed at the SAME full 8,000-path daily-rebalancing scale as the headline result (Part 3.4's COS-method upgrade is what made this possible; it previously ran at only ~150 paths). A warning-styled callout explains the counter-intuitive finding: Heston hedging is WORSE than even naive Black-Scholes on both assets, because stochastic volatility (Heston's specialty) is simply the wrong tool against jump risk (Merton's specialty) — "more sophisticated" is not the same as "better suited to the actual risk present."

Tab 10 — Vega Hedging

Implements Part 5.4. A precomputed, report-scale bar chart of Delta-only vs. Delta-Vega hedge CVaR (true market: each asset's own calibrated Heston process), followed by a fully live re-simulation section: choose the asset, a TRUE market (stochastic volatility OR jumps), the second instrument's strike, the rebalancing frequency, and the path count, and see both the resulting P&L distributions (as violin plots) and the CVaR comparison update immediately.

LOOK FOR

Switching the "true market" control between the two options: Delta-Vega hedging is built to help against stochastic-volatility risk specifically, and the live chart lets you see, on demand, that it does noticeably less (or even slightly harms) when the true risk is jumps instead — a hands-on version of the honest, nuanced finding from Part 5.4.

Tab 11 — Efficient Frontier & Bands

Two linked studies. First, Part 5.2's rebalancing-frequency sweep: a dual-axis chart of CVaR and standard deviation of P&L against rebalancing frequency, for both assets. Second, Part 5.3's band-hedging sweep: two scatter charts (a precomputed report-scale version and a fully live, re-sweepable version) plotting CVaR against average transaction cost paid for several tolerance-band widths, with a dashed reference line marking plain daily rebalancing's CVaR for comparison.

LOOK FOR

Points that sit **BELOW AND TO THE LEFT** of the dashed daily-rebalancing reference line — those band widths achieve **LOWER** risk at **LOWER** cost simultaneously, the "free lunch" result described in Part 5.3.

Tab 12 — American Options

Implements Part 9. Choose an asset and option type (put/call), and see: four metric cards (European tree price, American tree price, early-exercise premium, and the Crank-Nicolson PDE price alongside how closely it agrees with the tree), the early-exercise boundary curve traced directly from the PDE grid over the option's whole life, and a second chart of how the early-exercise premium itself changes across a range of strikes (moneyness).

LOOK FOR

The early-exercise boundary chart always staying below the strike price line for a put (Part 9.3) — if it ever crossed above, that would indicate a bug, since exercising a put when the stock is **ABOVE** the strike is never rational.

Tab 13 — Stress Testing

Implements Part 11.2. Choose an asset and option type, and see a bar chart of terminal hedging P&L for BS vs. Merton hedge across all six named scenarios, plus a summary noting how many of the scenarios the Merton hedge won outright, and a full numeric table.

Tab 14 — Model Validation

Implements Part 11.3. Four colour-coded cards (green/yellow/red, matching the Basel traffic light) showing each asset/hedge-model combination's out-of-sample exception rate and Kupiec p-value, a bar chart of exception rates against a dashed 5%-target reference line, and a "Live re-backtest" section where you can choose the asset, path count, VaR level, and train/test split fraction and re-run the entire honest out-of-sample test on demand.

Tab 15 — Interactive Risk Lab

A free-form exploration tab with no fixed narrative: choose an asset, option type, path count, transaction cost, random seed, and risk-free rate, and watch TWO simultaneous multi-parameter sweeps update — CVaR against rebalancing frequency (line chart AND a colour-coded heatmap) and CVaR against strike/moneyness — plus a "drill-down" section letting you pick one single illustrative simulated path and watch its underlying price, hedge Delta, and cash account evolve together over the option's life, side by side.

LOOK FOR

This tab is the best place to build your own intuition: nothing here is precomputed, so every control genuinely changes what gets simulated and shown.

Tab 16 — Methodology & Notes

A written reference tab (no charts) covering the same methodological choices, limitations, and engineering notes as the written report (this guide's companion document, `report/report.md`) — including a "Skills Demonstrated" table mapping each source file to the quantitative finance / numerical methods / statistics topic it showcases, useful as a quick index into the rest of this guide.

Headline Findings, Glossary, and References

A one-page summary of everything this project concluded, followed by a complete glossary of every term used in this guide.

14.1 The Whole Project, in Six Findings

1. **A market maker hedging with Black-Scholes Delta carries meaningfully more tail risk than one hedging with jump-aware Merton Delta**, on both a calm equity index (SPY, +22% higher CVaR) and a volatile cryptocurrency (BTC-USD, +22% higher CVaR in simulation, +64% in a real-history walk-forward backtest) — and this gap is statistically significant well beyond conventional thresholds ($p < 0.001$, Part 7).
2. **Stochastic volatility (Heston) is not a jump hedge**. Adding a "more sophisticated" Heston hedge, Delta-only, performed WORSE than plain Black-Scholes on both assets — sophistication has to match the specific risk present (jumps), not just add complexity in an unrelated dimension (Part 9's finding, confirmed at full statistical scale by the COS method upgrade).
3. **Giving Heston a Vega hedge helps — sometimes**. A Delta-Vega hedge cut CVaR by roughly a quarter for BTC-USD (genuine vol-of-vol risk) but slightly HURT for SPY (negligible vol-of-vol risk) — the same "match sophistication to the actual risk" lesson, one level deeper (Part 5.4).
4. **The finding survives independent, distribution-free corroboration**. Greeks-based P&L attribution, six named deterministic crash/rally scenarios, and out-of-sample VaR backtesting (Kupiec/Basel) all independently confirm the same qualitative conclusion using completely different methodologies (Part 11).
5. **A genuine look-ahead bias was found in the real-data backtest and fixed**. An earlier version calibrated using an asset's entire history to hedge windows from years earlier — a textbook methodological error. The corrected, walk-forward version still confirms the same conclusion, now honestly (Part 4.1, Part 12 Step 6).

6. **Extreme Value Theory, bootstrap confidence intervals, American options, and Monte Carlo variance reduction all extend the core finding's reach** — into deeper tail quantiles than the raw sample can estimate directly, into formal uncertainty quantification, into early-exercisable contracts, and into more computationally efficient rare-event estimation, respectively (Parts 7, 8, 9, 10).

14.2 Glossary

American option

An option that can be exercised at any time up to and including its maturity date, not just at maturity (Part 9).

Antithetic variates

A Monte Carlo variance-reduction technique that pairs each simulated random path with its exact mirror image (Part 10.2).

Basel traffic light

A regulatory framework classifying a VaR model as Green/Yellow/Red based on how many times it was breached out-of-sample (Part 11.3).

Bootstrap

A statistical technique for estimating an estimate's own uncertainty by repeatedly resampling, with replacement, from the data you already have (Part 7.2).

Calibration

Choosing numerical values for a model's parameters so it matches a specific real asset's behaviour (Part 4).

Call option

A contract giving the right, but not the obligation, to BUY the underlying at a fixed strike price (Part 2.1).

Cash account

The running ledger of cash a market maker holds while hedging, tracking premium received, share purchases/sales, transaction costs, and interest earned (Part 5.1).

Characteristic function

A mathematical "fingerprint" of a probability distribution (built via a Fourier transform) that can recover any probability of interest via an inversion formula, even when the distribution has no simple closed-form shape (Part 3.3).

Coherence (of a risk measure)

A property (satisfied by CVaR, not always by VaR) guaranteeing that combining two positions can never make your measured risk go up relative to holding them separately (Part 6.3).

Compound Poisson process

A process where random "events" (jumps) arrive at random times, and each event additionally has its own random size (Part 3.5).

Confidence interval

A range of values, built from a bootstrap or other statistical procedure, that we can be reasonably confident (e.g., 95% confident) contains the true, unknown value (Part 7.2).

Control variates

A Monte Carlo variance-reduction technique that corrects an estimate using a correlated quantity whose true answer is already known exactly (Part 10.3).

COS method

The Fourier-Cosine method (Fang & Oosterlee, 2008): a fast, vectorizable way to price options under Heston by approximating the payoff and probability distribution as sums of cosine waves (Part 3.4).

CVaR (Conditional Value-at-Risk)

The average loss across the worst q -fraction of outcomes — this project's primary risk measure (Part 6.3).

Delta (Δ)

An option's sensitivity to a \$1 move in the underlying price; also, the number of shares of the underlying to hold per option sold in order to hedge (Part 2.3, Part 3.2).

Delta-Vega hedge

A hedge using a second option instrument, sized to neutralize net Vega, in addition to the underlying (Part 5.4).

Diffusion / diffusive

The smooth, continuous (non-jumping) part of a price's random motion.

Drift

The predictable, average trend component of an asset's price motion (Part 2.4).

European option

An option that can only be exercised on its exact maturity date (Part 9.1).

Extreme Value Theory (EVT)

A branch of statistics for estimating rare, extreme outcomes using a universal limiting distribution (the GPD) rather than raw historical extremes alone (Part 8).

Finite difference

Estimating a derivative/sensitivity numerically by nudging an input up and down by a tiny amount and dividing the resulting price difference by the nudge size (Part 3.2); also, the general numerical-methods family the Crank-Nicolson PDE solver belongs to (Part 9.3).

Gamma (Γ)

The rate at which Delta itself changes as the underlying price moves (Part 3.2).

Generalized Pareto Distribution (GPD)

The universal limiting shape of extreme exceedances over a high threshold, per the Pickands–Balkema–de Haan theorem (Part 8.2).

Geometric Brownian Motion (GBM)

The standard "drift plus random wiggle" model of continuous price motion underlying Black-Scholes (Part 2.4).

Heston model

An option-pricing model in which volatility itself is a second, random, mean-reverting process (Part 3.3).

Hill estimator

A simpler, older estimator of a distribution's tail shape parameter, exact for pure power-law tails but biased for Generalized Pareto tails (Part 8.3).

Implied volatility

The volatility number that, when plugged into Black-Scholes, reproduces an observed (market or model) option price.

Importance sampling

A Monte Carlo technique that deliberately samples from a "wrong," tilted distribution emphasizing rare/important outcomes, then mathematically reweights the results to remove the bias this introduces (Part 10.4).

Jump / jump-diffusion

A sudden, discontinuous change in an asset's price, as opposed to smooth, continuous drift-and-wiggle motion (Part 2.5, Part 3.5).

Kupiec's test

A likelihood-ratio statistical test of whether a VaR model's observed breach rate matches its stated target rate (Part 11.3).

Leverage effect

The empirically-observed tendency for volatility to rise when prices fall (Part 3.3).

Look-ahead bias

A methodological error where a model or backtest uses information that would not actually have been available at the time being studied (Part 4.1, Part 12 Step 6).

Market maker

A firm that stands ready to buy and sell an asset (here, options) continuously, profiting from the bid-ask spread rather than directional bets (Part 2.2).

Maturity / expiry (T)

The date an option contract stops existing (Part 2.1).

Mean reversion

The tendency of a quantity (e.g., volatility) to drift back toward its long-run average after a shock (Part 3.3).

Merton (jump-diffusion) model

An option-pricing model that adds sudden random jumps to otherwise ordinary Geometric Brownian Motion (Part 3.5).

Monte Carlo simulation

Estimating a mathematical quantity by generating and averaging over a large number of random simulated scenarios (Part 10.1).

P-measure (physical measure)

Probabilities/calibration based on how an asset has ACTUALLY behaved historically (Part 4.1).

Payoff

What an option is worth at maturity, e.g. $\max(S_T - K, 0)$ for a call (Part 2.1).

P&L attribution

Decomposing a change in an option's value into pieces explained by each Greek, plus a residual (Part 11.1).

Poisson process

A standard probability model for randomly-timed rare events, characterized by an arrival rate λ (Part 3.5).

Premium

The price paid today to own an option (Part 2.1).

p-value

The probability of observing a result at least as extreme as what was actually seen, if there were truly no real underlying effect (Part 7.4).

Put option

A contract giving the right, but not the obligation, to SELL the underlying at a fixed strike price (Part 2.1).

Q-measure (risk-neutral measure)

Probabilities/calibration based on what current, live option market prices imply (Part 4.1).

Quantile

The value below which a given fraction of a sorted dataset falls (Part 6.2).

Rebalancing

Adjusting a hedge position (buying/selling shares) to keep it aligned with the option's current Delta (Part 5.2).

Rho (ρ as a Greek)

An option's sensitivity to the risk-free interest rate (Part 3.2).

Standard error

A measure of how much an estimate (such as a Monte Carlo average) would plausibly vary if the underlying random sampling were repeated (Part 7.1, Part 10.1).

Statistical significance

A result unlikely to have occurred by chance alone, conventionally when its p-value is below 0.05 (Part 7.4).

Stress testing

Running specific, hand-designed adverse scenarios through a model, independent of any probability distribution (Part 11.2).

Strike (K)

The fixed price written into an option contract (Part 2.1).

Theta (Θ)

An option's rate of value decay purely from time passing (Part 3.2).

Threshold (in EVT)

The high cutoff loss level above which the Generalized Pareto Distribution is fitted to exceedances (Part 8.2).

Underlying (S)

The actual asset (e.g., SPY, BTC-USD) that an option is written on (Part 2.1).

VaR (Value-at-Risk)

The loss level exceeded only q -fraction of the time (a quantile of the loss distribution) (Part 6.2).

Variance reduction

Any technique that reduces a Monte Carlo estimate's standard error without needing more simulated paths (Part 10).

Vega

An option's sensitivity to a change in volatility (Part 3.2).

Volatility (σ)

The annualized typical size of an asset's random price fluctuations (Part 2.4).

Walk-forward calibration

Recalibrating a model repeatedly through time, using only data available up to each point, to avoid look-ahead bias (Part 4.1).

14.3 References

- Albrecher, H., Mayer, P., Schoutens, W., & Tistaert, J. (2007). "The little Heston trap." *Wilmott Magazine*.
- Balkema, A.A., & de Haan, L. (1974). "Residual life time at great age." *Annals of Probability*, 2(5), 792–804.
- Black, F., & Scholes, M. (1973). "The pricing of options and corporate liabilities." *Journal of Political Economy*, 81(3), 637–654.
- Cox, J.C., Ross, S.A., & Rubinstein, M. (1979). "Option pricing: a simplified approach." *Journal of Financial Economics*, 7(3), 229–263.
- Crank, J., & Nicolson, P. (1947). "A practical method for numerical evaluation of solutions of PDEs of the heat-conduction type." *Proc. Cambridge Phil. Soc.*, 43(1).
- Efron, B., & Tibshirani, R.J. (1993). *An Introduction to the Bootstrap*. Chapman & Hall.
- Fang, F., & Oosterlee, C.W. (2008). "A novel pricing method for European options based on Fourier-cosine series expansions." *SIAM J. Sci. Comput.*, 31(2), 826–848.
- Glasserman, P. (2003). *Monte Carlo Methods in Financial Engineering*. Springer.
- Heston, S.L. (1993). "A closed-form solution for options with stochastic volatility." *Review of Financial Studies*, 6(2), 327–343.
- Hill, B.M. (1975). "A simple general approach to inference about the tail of a distribution." *Annals of Statistics*, 3(5), 1163–1174.
- Hull, J.C. *Options, Futures, and Other Derivatives*. Pearson (textbook benchmark used in Part 3.1).
- Kupiec, P. (1995). "Techniques for verifying the accuracy of risk measurement models." *Journal of Derivatives*, 3(2).
- Lord, R., Koekkoek, R., & Van Dijk, D. (2010). "A comparison of biased simulation schemes for stochastic volatility models." *Quantitative Finance*, 10(2), 177–194.
- McNeil, A.J., & Frey, R. (2000). "Estimation of tail-related risk measures for heteroscedastic financial time series." *Journal of Empirical Finance*, 7(3-4).
- Merton, R.C. (1976). "Option pricing when underlying stock returns are discontinuous." *Journal of Financial Economics*, 3(1-2), 125–144.
- Pickands, J. (1975). "Statistical inference using extreme order statistics." *Annals of Statistics*, 3(1), 119–131.
- Whalley, A.E., & Wilmott, P. (1993). "An asymptotic analysis of an optimal hedging model for option pricing with transaction costs." *Mathematical Finance*.
- Wilmott, P., Howison, S., & Dewynne, J. (1995). *The Mathematics of Financial Derivatives*. Cambridge University Press.
-

This guide is a companion to `README.md` and `report/report.md` in the project repository, which contain the complete, condensed technical write-up and every result table this guide references.